

SMT-Tutorial-for-AI-Class-1

May 7, 2026

1 Statistical Machine Translation Tutorial (Intro)

Ye Kyaw Thu, Lab Leader, Language Understanding Lab., Myanmar

For AI Fundamental Class students

Date: 26 April 2026

ဒီ Notebook က Github က MTRSS (Machine Translation Research Summer School) က code တွေကို ဆာဗာတစ်ခုပေါ်ကို ဒေါင်းလုဒ်လုပ်ပြီး အတန်းထဲမှာ ပထမဆုံး run ပြခဲ့တာမို့ error သုံးခု တက်တာကိုလည်း လက်တွေ့ရှင်းရင်းနဲ့ run ခဲ့တဲ့ notebook ပါ။

- ပထမ error က ထုံးစံအတိုင်းပဲ perl script တွေ၊ configuration file မှာ ကိုယ့်စက်ရဲ့ path တွေကို ဖြည့်တဲ့အခါမှာ ကျန်ခဲ့တာမျိုးပါ
- ဒုတိယ error က GIZA++ နဲ့ ပတ်သက်ပြီး ဖြစ်တဲ့ error ပါ။ GIZA++ install လုပ်ထားတဲ့ path (/home/ye/tool/giza-pp/GIZA++-v2) အောက်မှာ mklcls ဖိုင်ကိုပါ ကော်ပီကူးထည့်လိုက်ရင် ပြေလည်းပါတယ်။
- တတိယ error ကတော့ SGM အတွက် ဆောက်ထားတဲ့ ဖိုလ်ဒါနာမည်ကို စကားပြောရင်း၊ စာသင်ရင်းနဲ့ ရိုက်လိုက်တာမို့ test-gen ဆိုပြီး မှာရိုက်လိုက်လို့ SMT run တဲ့အခါမှာ တက်တဲ့ error ပါ။ အမှန်က test-
sgm ဖြစ်ရပါမယ်။ :P

1.1 Data

```
[1]: %pwd
```

```
[1]: '/home/ye/exp/SMT-NMT_tutorial'
```

```
[2]: !ls ./data/*
```

```
./data/dev.my    ./data/test.my    ./data/train.my  
./data/dev.th    ./data/test.th    ./data/train.th
```

1.2 Data Size

for training:

```
[4]: !wc ./data/train.*
```

```

62625    734742  7067941 ./data/train.my
62625    439436  5110712 ./data/train.th
125250   1174178 12178653 total

```

for development or validation:

```
[5]: !wc ./data/dev.*
```

```

10000   117884 1133121 ./data/dev.my
10000    71013  825957 ./data/dev.th
20000   188897 1959078 total

```

for testing:

```
[6]: !wc ./data/test.*
```

```

8000    93919  901455 ./data/test.my
8000    56113  651909 ./data/test.th
16000   150032 1553364 total

```

Let's see data format:

```
[7]: !head ./data/train.th
```

```
[8]: !head ./data/train.my
```

```

အောက် ပါ ရော ဂါ လက္ခဏာ တွေ များ ရှိ ရင်
စိတ် ကို အေး အေး ဆေး ဆေး ထား ပါ
နာ တာ ရှည် ရော ဂါ ရယ် လို့ မ ရှိ ပါ ဘူး ဗျ
လက္ခဏာ တွေ နဲ့
ပြီး တဲ့ အ ခါ နား ကြပ် ကို တံ တော ငဲ့ ဆစ် နား မှာ ထား မှာ မို့ ပါ ရှင်
ပု စွန် တော့ ကျွန် တော် စား တယ်
သို့ ပေ မဲ့ လည်း
ကျွန် တော် ခေါင်း ပေါ် ရေ ခဲ တင် ထား ရ မ လား
ကွန် ပြူ တာ ရောင် ခြည် နဲ့ ထပ် စစ် ဆေး ပါ မယ် နော်
ဒီ ကြား မှာ ခ ဏ လေး ထိုင် စော ငဲ့ ပေး လို့ ရ မ လား

```

```
[9]: !head ./data/dev.th
```

[10]: !head ./data/dev.my

ဒီ လို မျိုး ဘယ် ချိန် က စ ဖြစ် တာ လဲ
အ စာ စား ပြီး တိုင်း အ သက် ရှူ ရင် မ ဝ သ လို ခံ စား ရ တယ်
ဒီ လို အ ဖြစ် အ ပျက် မျိုး ထပ် မ ကြုံ ရ အောင် ပေါ့
ငါး လှီး တဲ့ ဓါး
လည် ချောင်း လည်း နာ ကျင် တယ် ရှင်
ဘယ် နား ကိုက် သွား တာ လဲ ရှင်
တစ် အော င် နေ သွား တိုက် တာ သင် တဲ့ ဗီ ဒီ ယို ကို ကြည့် ပါ ခင် ဗျာ
မျက် လုံး မှိတ် ပြ ပါ
ခွဲ စိတ် လို့ ပြီး သွား ရင် ပို မို နာ ကျင် နိုင် လိ မ့် မယ်
ဆ ရာ မ အ တွင်း ဘက် ကို တ ဆက် ထဲ စမ်း သပ် မယ် ရှင်

[11]: !head ./data/test.th

[12]: !head ./data/test.my

ခွဲ မှ ရ မှာ လား
မျက် ခံ လှုပ် တဲ့ လက္ခဏာ ရှိ မယ်
အ ရက် ကို စွ န် လွှတ် ရ မည် ဖြစ် ပြီး ပ ရှိ တင်း နှ င် ဗီ တာ မင် ဓာတ် ကြွယ် ဝ
သော အာ ဟာ ရ ရှိ သော အ စား အ စာ များ ကို စား သ င် သည်
ဟုတ် ကဲ့ သူ ဖျား ပါ တယ် သူ သီး သန့် တော့ မ နေ ခဲ့ ဘူး
မောင် လေး နောက် ဆုံး အ ကြိမ် ဆေး သောက် ခဲ့ တာ ဘယ် ချိန် လဲ
ကိုယ် မှာ ကူး စက် တာ က နေ ဦး နောက် ထိ ပါ ကူး စက် လာ တယ်

မစ် စ တာ ပ ရာ မို့ ရဲ့ ဆွေ မျိုး ဝင် လာ ဖို့ တစ် ချက် ခေါ် ပေး ပါ ရှင်
 အ ချို့ ကို လျှော့ ချ ကြ ည့် လိုက် ပါ
 သူ တို့ နဲ့ အ တူ တူ ရှိ နေ တယ် လို့ ပဲ တွေး မိ နေ တယ် ဗျာ
 ဆေး ယူ ပြီး ပြီ ဆို ရင် တော့ အိမ် ပြန် လို့ ရ ပြီ နော်

1.3 Data Cleaning

မနေ့ကအတန်းမှာ ရှင်းပြခဲ့သလိုပဲ။ မြန်မာစာကို `sylbreak.py` နဲ့ syllable breaking လုပ်တဲ့အခါမှာ စာရိုက်ထားတဲ့ အစီအစဉ် (typing order) က မမှန်တာမျိုးတွေကို တွေ့ရတယ်။ ဥပမာ “စွ န်”, “ကြ ည့်”

အဲဒါမျိုး မဖြစ်အောင် အရင်ဆုံး မြန်မာစာ ဒေတာတွေကို cleaning လုပ်ကြရအောင်။

မလုပ်ရင် machine translation performance ကို အနည်းနဲ့အများ သက်ရောက်မှုရှိလိမ့်မယ်။

ပြီးတော့ ထိုင်း ကနေ မြန်မာစာကို ဘာသာပြန်ခိုင်းပြီး ထွက်လာတဲ့ output စာကြောင်းတွေမှာလည်း အမှားတွေပါနေလိမ့်မယ်။

အရင်ဆုံး ဘယ်လိုမှားနေတယ် ဆိုတာကို မြင်သာအောင် အထက်က မှားနေတဲ့ စာလုံး နှစ်လုံးကိုပဲ သုံးပြီး အမှန် ရိုက်ထားတာနဲ့ နှိုင်းယှဉ်ကြည့်ကြရအောင်။

text file တစ်ဖိုင်အနေနဲ့ အောက်ပါ စာကြောင်းတွေကို သိမ်းပါ။ အမှားက အပေါ်က၊ မှန်မှန်ကန်ကန် ရိုက်ထည့်ထားတာက အောက်က syllable ပါ။

စွန်
 စွန်
 ကြည့်
 ကြည့်

```
[17]: % bash
cat <<EOF > error-eg.txt
စွန်
စွန်
ကြည့်
ကြည့်
EOF
```

မိုင် အဖြစ် သိမ်းမသိမ်းကို confirm လုပ်ကြည့်ပါ။

```
[18]: !cat ./error-eg.txt
```

စွန်
 စွန်
 ကြည့်
 ကြည့်

tool/ folder အောက်မှာ Unicode number တွေအနေနဲ့ print ထုတ်ပေးအောင် ရေးထားတဲ့ perl script ရှိတယ်။ အဲဒါကို သုံးပြီး အထက်က မြင်နေရတဲ့ မြန်မာစာ စာလုံးတွေကို ရိုက်ထုတ်ခိုင်းကြည့်ရအောင်။ အဲဒါဆိုရင် မှားတဲ့ syllable စာလုံးနဲ့ မှန်တဲ့ syllable စာလုံးအကြားမှာ typing order က ဘယ်လိုဖြစ်နေသလဲ ဆိုတာကို မြင်ရပါလိမ့်မယ်။ လေ့လာကြည့်ပါ။

```
[19]: !perl ./tool/print-codepoint.pl ./error-eg.txt
```

```
စွန့်
စ (4101, U1005) ဝ (4157, U103d) န (4116, U1014) ဝ (4151, U1037) န် (4154, U103a)
, no. of char = 5
စွန့်
စ (4101, U1005) ဝ (4157, U103d) န (4116, U1014) န် (4154, U103a) ဝ (4151, U1037)
, no. of char = 5
ကြည့်
က (4096, U1000) ိ (4156, U103c) ည (4106, U100a) ဝ (4151, U1037) န် (4154, U103a)
, no. of char = 5
ကြည့်
က (4096, U1000) ိ (4156, U103c) ည (4106, U100a) န် (4154, U103a) ဝ (4151, U1037)
, no. of char = 5
```

1.4 Notes

အထက်မှာ မြင်ရတဲ့အတိုင်းပဲ။ ကိုယ့်ပိုင်ထဲမှာ ရိုက်ထားတဲ့ syllable တွေရဲ့ order က မှားနေရင်တော့ sylbreak tool နဲ့ syllable segmentation လုပ်တဲ့အခါမှာ မှားဖြစ်ပေးပါလိမ့်မယ်။ pipe ကို delimiter အဖြစ်ထားပြီး ဖြတ်ရင် အောက်ပါလိုမျိုး ရလဒ် ရလိမ့်မယ်...

```
[20]: !python ./tool/sylbreak.py --input ./error-eg.txt
```

```
စ | န်
စွန့်
ကြ | ည့်
ကြည့်
```

1.5 Normalization

အထက်ပါလိုမျိုး typing order အပြင်ကို တကယ်က spelling mistake, translation error စတာမျိုးတွေလည်း အများကြီး ပါနိုင်ပါတယ်။

ဒီ tutorial မှာတော့ syllable level အမှားတွေကိုပဲ ပြင်တဲ့အပိုင်း တနည်းအားဖြင့် syllable normalization အပိုင်းကိုပဲ data cleaning ဥပမာ အနေနဲ့ လုပ်ပြပါမယ်။

အရင်ဆုံး လက်ရှိ data folder တစ်ခုလုံးကို ကော်ပီကူးပြီး အဲဒီအထဲမှာပဲ normalization process ကို လုပ်ကြရအောင်။

မဟုတ်ရင် Myanmar-Thai parallel corpus ဖိုင်တွေကို မှားပြီး ဖျက်မိတာ၊ ပြင်မိတာမျိုးတွေ ရှိရင် ပြန်ရဖို့ ခက်တာမို့။

```
[21]: !cp -r ./data ./clean-data
```

```
[22]: !ls ./clean-data/*
```

```
./clean-data/dev.my ./clean-data/test.my ./clean-data/train.my  
./clean-data/dev.th ./clean-data/test.th ./clean-data/train.th
```

```
[23]: %cd ./clean-data/
```

```
/home/ye/exp/SMT-NMT_tutorial/clean-data
```

```
[24]: %mkdir normalize
```

```
[27]: !mv *.* ./normalize
```

```
[28]: !tree
```

```
.  
  normalize  
    dev.my  
    dev.th  
    test.my  
    test.th  
    train.my  
    train.th
```

```
2 directories, 6 files
```

အသစ်ဆောက်ထားတဲ့ normalize/ folder အောက်မှာပဲ မှားနေတဲ့ syllable တွေကို ပြင်တာတွေ လုပ်မယ်။ ပြီးမှ data-clean/ အောက်ကို ကော်ပီပြန်ကူးယူကြမယ်။

Normalization tool က ဆရာတို့ Lab က develop လုပ်နေတုန်းပါပဲ။ လက်ရှိ ဗားရှင်းကိုပဲ သုံးကြရအောင်။

```
[29]: %cd ..
```

```
/home/ye/exp/SMT-NMT_tutorial
```

```
[31]: !ls ./syl_normalizer/
```

```
final_syl_dictionary_13Feb2024.sorted.txt  real_err1.syl.normalized  
note.txt                                  syllable_checked_result.txt  
process_dictionary.py                     syl_normalizer.py  
real_err1.syl
```

syl_normalizer.py code ကို လေ့လာကြည့်ရအောင်။

```
[53]: !cat ./syl_normalizer/syl_normalizer.py
```

```
"""
```

```
RE rules and syllable dictionary based normalization for Burmese.
```

```
Note: Still developing and to clean real-word errors
```

```
when need to add some more rules and also syllables based on your working
```

domain.

Written by Ye Kyaw Thu, LU Lab., Myanmar.

Previous updated: 14 Feb 2024

Last updated: 26 April 2026

Usage:

```
time python3 ./syl_normalizer.py \  
--dictionary ./final_syl_dictionary_13Feb2024.sorted.txt --frequency 2 \  
--input real_err1.syl --out real_err1.syl.normalized
```

"""

```
import re  
import argparse  
import sys
```

```
class BurmeseSyllableChecker:
```

```
    def __init__(self, dictionary_file, min_frequency=2):
```

```
        C = r'[က-အ]' # Consonants
```

```
        M = r'(့|့)(့)?' # Medials
```

```
        V = r'ေ?(?!ီ|ီ)(ါ|ာ)?(ိ|ိ|ီ|ဲ)?(ု|ူ)?' # Vowels
```

```
        F = r'[ံ?]' # Final consonants and signs
```

```
        A = r'ိ' # Asat
```

```
        S = r'ာ' # Sa-lon
```

```
        G = r'ယ' # Great SA
```

```
        IVS = r'([ကုဉ္ဇိဓတြာဉ္ဇိဉ်၏ငါး])?' # Independent Vowels before
```

```
↳Subscripts
```

```
        SUB = r'((?:{C}{S}{C})+).format(C=C, S=S) # Subscript syllables,  
allowing for multiple
```

```
        KZ = r'ိ'
```

```
# Merging the Complex pattern for better handling of various cases
```

```
Complex = r '(' \  
    f"{IVS}?" \  
    f"(?:{C}{M}?{V}?{F}?)*" \  
    f"(?:{SUB}{V}?{F}?)*" \  
    f"|{G}{V}?" \  
    f"|{KZ}{C}{S}{C}(?:{V}?{F}?)*" \  
    f"|{C}{S}{C}{V}{F}?" \  
    f"|{C}{M}?{V}?{F}?{A}?" \  
    f"|{C}{C}{A}{C}{M}{A}" \  
    f"|{SUB}{C}{A}" \  
    f"|{C}{M}{M}{C}{A}{V}{C}{A}" \  
    f"|{C}{SUB}{C}{A}{F}?" \  
    f"|{IVS}{SUB}{C}" \  
    f"|{C}{SUB}*{V}{C}{A}{F}?" \
```

```

        f"|{C}{KZ}{C}{SUB}" \
        r')'

self.patterns = {
    'C': C,
    'M': M,
    'V': V,
    'F': F,
    'A': A,
    'S': S,
    'G': G,
    'I': r'[ဤဧဥဦဩဦ၏]', # Independent vowels
    'E': r'[ဣဥဦဩဦ]', # Independent vowels with "အ"
    'D': r'[၀-၉]', # Digits
    'P': r'[။]', # Punctuation
    'SUB': SUB,
    'IVS': IVS,
    'Complex': Complex,
    'K': r'({C}{M}?{KZ}{C}{M}?{V}?{A}?[ံ့း]?({C}{A}း?))'.format(C=C,
M=M, V=V, A=A, KZ=KZ),
    'CA': r'({C}{M}{V}{F}?{C}{A}း?)'.format(C=C, M=M, V=V, F=F, A=A)
}

self.syllable_regex = re.compile(
    f"^(
    f"{Complex}|"
    f"{IVS}{C}{M}{V}{F}{A}?"
    f"{self.patterns['E']}{C}?{A}?{F}|"
    f"{self.patterns['I']}|"
    f"{self.patterns['D']}|"
    f"{self.patterns['P']}|"
    f"{self.patterns['CA']}|"
    f"{self.patterns['K']}
    f")$",
    re.UNICODE
)

self.dictionary = {}
with open(dictionary_file, 'r', encoding='utf-8') as f:
    for line in f:
        syllable, frequency = line.strip().split()
        frequency = int(frequency)
        if frequency >= min_frequency:
            self.dictionary[syllable] = frequency

def check_syllable(self, syllable):
    if self.syllable_regex.fullmatch(syllable) and syllable in

```

```

self.dictionary:
    #if syllable in self.dictionary:
        return syllable
    else:
        return f"<{syllable}>"

class BurmeseNormalizer:
    def __init__(self):
        self.rules = [
            # Rule 1 (Updated)
            {'pattern': r'[\u200B\u200C\u202C\u00A0\u200D\u200A]', 'repl': ''},
            # Rule 2
            {'pattern': r'([\u0300\u0301\u0302\u0303\u0304\u0305\u0306\u0307\u0308\u0309\u030A\u030B\u030C\u030D\u030E\u030F\u0310\u0311\u0312\u0313\u0314\u0315\u0316\u0317\u0318\u0319\u031A\u031B\u031C\u031D\u031E\u031F\u0320\u0321\u0322\u0323\u0324\u0325\u0326\u0327\u0328\u0329\u032A\u032B\u032C\u032D\u032E\u032F\u0330\u0331\u0332\u0333\u0334\u0335\u0336\u0337\u0338\u0339\u033A\u033B\u033C\u033D\u033E\u033F\u0340\u0341\u0342\u0343\u0344\u0345\u0346\u0347\u0348\u0349\u034A\u034B\u034C\u034D\u034E\u034F\u0350\u0351\u0352\u0353\u0354\u0355\u0356\u0357\u0358\u0359\u035A\u035B\u035C\u035D\u035E\u035F\u0360\u0361\u0362\u0363\u0364\u0365\u0366\u0367\u0368\u0369\u036A\u036B\u036C\u036D\u036E\u036F\u0370\u0371\u0372\u0373\u0374\u0375\u0376\u0377\u0378\u0379\u037A\u037B\u037C\u037D\u037E\u037F\u0380\u0381\u0382\u0383\u0384\u0385\u0386\u0387\u0388\u0389\u038A\u038B\u038C\u038D\u038E\u038F\u0390\u0391\u0392\u0393\u0394\u0395\u0396\u0397\u0398\u0399\u039A\u039B\u039C\u039D\u039E\u039F\u03A0\u03A1\u03A2\u03A3\u03A4\u03A5\u03A6\u03A7\u03A8\u03A9\u03AA\u03AB\u03AC\u03AD\u03AE\u03AF\u03B0\u03B1\u03B2\u03B3\u03B4\u03B5\u03B6\u03B7\u03B8\u03B9\u03BA\u03BB\u03BC\u03BD\u03BE\u03BF\u03C0\u03C1\u03C2\u03C3\u03C4\u03C5\u03C6\u03C7\u03C8\u03C9\u03CA\u03CB\u03CC\u03CD\u03CE\u03CF\u03D0\u03D1\u03D2\u03D3\u03D4\u03D5\u03D6\u03D7\u03D8\u03D9\u03DA\u03DB\u03DC\u03DD\u03DE\u03DF\u03E0\u03E1\u03E2\u03E3\u03E4\u03E5\u03E6\u03E7\u03E8\u03E9\u03EA\u03EB\u03EC\u03ED\u03EE\u03EF\u03F0\u03F1\u03F2\u03F3\u03F4\u03F5\u03F6\u03F7\u03F8\u03F9\u03FA\u03FB\u03FC\u03FD\u03FE\u03FF\u0400\u0401\u0402\u0403\u0404\u0405\u0406\u0407\u0408\u0409\u040A\u040B\u040C\u040D\u040E\u040F\u0410\u0411\u0412\u0413\u0414\u0415\u0416\u0417\u0418\u0419\u041A\u041B\u041C\u041D\u041E\u041F\u0420\u0421\u0422\u0423\u0424\u0425\u0426\u0427\u0428\u0429\u042A\u042B\u042C\u042D\u042E\u042F\u0430\u0431\u0432\u0433\u0434\u0435\u0436\u0437\u0438\u0439\u043A\u043B\u043C\u043D\u043E\u043F\u0440\u0441\u0442\u0443\u0444\u0445\u0446\u0447\u0448\u0449\u044A\u044B\u044C\u044D\u044E\u044F\u0450\u0451\u0452\u0453\u0454\u0455\u0456\u0457\u0458\u0459\u045A\u045B\u045C\u045D\u045E\u045F\u0460\u0461\u0462\u0463\u0464\u0465\u0466\u0467\u0468\u0469\u046A\u046B\u046C\u046D\u046E\u046F\u0470\u0471\u0472\u0473\u0474\u0475\u0476\u0477\u0478\u0479\u047A\u047B\u047C\u047D\u047E\u047F\u0480\u0481\u0482\u0483\u0484\u0485\u0486\u0487\u0488\u0489\u048A\u048B\u048C\u048D\u048E\u048F\u0490\u0491\u0492\u0493\u0494\u0495\u0496\u0497\u0498\u0499\u049A\u049B\u049C\u049D\u049E\u049F\u04A0\u04A1\u04A2\u04A3\u04A4\u04A5\u04A6\u04A7\u04A8\u04A9\u04AA\u04AB\u04AC\u04AD\u04AE\u04AF\u04B0\u04B1\u04B2\u04B3\u04B4\u04B5\u04B6\u04B7\u04B8\u04B9\u04BA\u04BB\u04BC\u04BD\u04BE\u04BF\u04C0\u04C1\u04C2\u04C3\u04C4\u04C5\u04C6\u04C7\u04C8\u04C9\u04CA\u04CB\u04CC\u04CD\u04CE\u04CF\u04D0\u04D1\u04D2\u04D3\u04D4\u04D5\u04D6\u04D7\u04D8\u04D9\u04DA\u04DB\u04DC\u04DD\u04DE\u04DF\u04E0\u04E1\u04E2\u04E3\u04E4\u04E5\u04E6\u04E7\u04E8\u04E9\u04EA\u04EB\u04EC\u04ED\u04EE\u04EF\u04F0\u04F1\u04F2\u04F3\u04F4\u04F5\u04F6\u04F7\u04F8\u04F9\u04FA\u04FB\u04FC\u04FD\u04FE\u04FF\u0500\u0501\u0502\u0503\u0504\u0505\u0506\u0507\u0508\u0509\u050A\u050B\u050C\u050D\u050E\u050F\u0510\u0511\u0512\u0513\u0514\u0515\u0516\u0517\u0518\u0519\u051A\u051B\u051C\u051D\u051E\u051F\u0520\u0521\u0522\u0523\u0524\u0525\u0526\u0527\u0528\u0529\u052A\u052B\u052C\u052D\u052E\u052F\u0530\u0531\u0532\u0533\u0534\u0535\u0536\u0537\u0538\u0539\u053A\u053B\u053C\u053D\u053E\u053F\u0540\u0541\u0542\u0543\u0544\u0545\u0546\u0547\u0548\u0549\u054A\u054B\u054C\u054D\u054E\u054F\u0550\u0551\u0552\u0553\u0554\u0555\u0556\u0557\u0558\u0559\u055A\u055B\u055C\u055D\u055E\u055F\u0560\u0561\u0562\u0563\u0564\u0565\u0566\u0567\u0568\u0569\u056A\u056B\u056C\u056D\u056E\u056F\u0570\u0571\u0572\u0573\u0574\u0575\u0576\u0577\u0578\u0579\u057A\u057B\u057C\u057D\u057E\u057F\u0580\u0581\u0582\u0583\u0584\u0585\u0586\u0587\u0588\u0589\u058A\u058B\u058C\u058D\u058E\u058F\u0590\u0591\u0592\u0593\u0594\u0595\u0596\u0597\u0598\u0599\u059A\u059B\u059C\u059D\u059E\u059F\u05A0\u05A1\u05A2\u05A3\u05A4\u05A5\u05A6\u05A7\u05A8\u05A9\u05AA\u05AB\u05AC\u05AD\u05AE\u05AF\u05B0\u05B1\u05B2\u05B3\u05B4\u05B5\u05B6\u05B7\u05B8\u05B9\u05BA\u05BB\u05BC\u05BD\u05BE\u05BF\u05C0\u05C1\u05C2\u05C3\u05C4\u05C5\u05C6\u05C7\u05C8\u05C9\u05CA\u05CB\u05CC\u05CD\u05CE\u05CF\u05D0\u05D1\u05D2\u05D3\u05D4\u05D5\u05D6\u05D7\u05D8\u05D9\u05DA\u05DB\u05DC\u05DD\u05DE\u05DF\u05E0\u05E1\u05E2\u05E3\u05E4\u05E5\u05E6\u05E7\u05E8\u05E9\u05EA\u05EB\u05EC\u05ED\u05EE\u05EF\u05F0\u05F1\u05F2\u05F3\u05F4\u05F5\u05F6\u05F7\u05F8\u05F9\u05FA\u05FB\u05FC\u05FD\u05FE\u05FF\u0600\u0601\u0602\u0603\u0604\u0605\u0606\u0607\u0608\u0609\u060A\u060B\u060C\u060D\u060E\u060F\u0610\u0611\u0612\u0613\u0614\u0615\u0616\u0617\u0618\u0619\u061A\u061B\u061C\u061D\u061E\u061F\u0620\u0621\u0622\u0623\u0624\u0625\u0626\u0627\u0628\u0629\u062A\u062B\u062C\u062D\u062E
```

```

# Rule 19
{'pattern': r' ေ([က-အ]) ျ', 'repl': r'\1 ျေ'},
# Rule 20
{'pattern': r' တကသိုလ်', 'repl': ' တက္ကသိုလ်'},
# Rule 21
{'pattern': r'tတ်', 'repl': ' သတ်'},
# Rule 22
{'pattern': r' ေတ', 'repl': ' သေ'},
# Rule 23
{'pattern': r' ေsတ်', 'repl': ' စောက်'},
# Rule 24
{'pattern': r' ဩ', 'repl': ' ဩ'},
# Rule 25
{'pattern': r' ဩြ', 'repl': ' ဩြ'},
# Rule 26
{'pattern': r' င်း', 'repl': ' င်း'},
# Rule 27
{'pattern': r'([က-အ]) ိ', 'repl': r'\1 ိ'},
# Rule 28
{'pattern': r' ျ', 'repl': ' ျ'},
# Rule 29
{'pattern': r' ကန်', 'repl': ' ကန်'},
# Rule 30
{'pattern': r'([က-အ]) ိ', 'repl': r'\1 ိ'},
# Rule 31
{'pattern': r' ဆဉ်', 'repl': ' ဆင်'},
# Rule 33
{'pattern': r' ိး', 'repl': ' ိး'},
# Rule 34
{'pattern': r' ိး', 'repl': ' ိး'},
# Rule 35
{'pattern': r' ိး', 'repl': ' ိး'},
# Rule 36
{'pattern': r' ေ([၁၂၃]) ိ', 'repl': r'\1 ိ'},
# Rule 37
{'pattern': r' o([\u102B-\u103E])', 'repl': r'o\1'},
# Rule 38
{'pattern': r' (ေ-ွး|ခေး|ေ_ွး)', 'repl': 'ွေး'},
# Rule 39
{'pattern':
r'(?<![က-အ])(\u1004)(\u1031)(\u103A)(\u1039)([\u1000-\u1021])', 'repl':
r'\1\3\4\5\2'},
# New Rule 40
{'pattern': r'([က-အ])([က-အ])([ျ]) ိ', 'repl': r'\1\3\2 ိ'},

```

```

        # New Rule 41
        {'pattern': r'ေ([က-အ])', 'repl': r'\1ေ'},
        # New Rule 42
        {'pattern': r'္တ္တ', 'repl': r'္တ္တ'},
        # New Rule 43
        {'pattern': r'မ္မ', 'repl': r'မ္မ'},
    ]

def normalize(self, input_text):
    # Split input text into lines
    lines = input_text.split('\n')
    output_lines = []

    for line in lines:
        # Split each line into words based on spaces
        words = line.split()
        normalized_words = []

        for word in words:
            # Apply normalization rules to each word
            for rule in self.rules:
                word = re.sub(rule['pattern'], rule['repl'], word)

            normalized_words.append(word)

        # Join the normalized words back into a line
        normalized_line = ' '.join(normalized_words)
        output_lines.append(normalized_line)

    # Join the normalized lines back into a single string
    output_text = '\n'.join(output_lines)
    return output_text

def process_input(input_stream, output_file=None, verbose=False,
dictionary_file=None, min_frequency=2, check='dictionary'):
    checker = BurmeseSyllableChecker(dictionary_file, min_frequency)
    normalizer = BurmeseNormalizer()
    line_number = 0
    results = []

    for line in input_stream:
        line = line.strip()
        syllables = line.split()
        line_number += 1
        if check == 'RE_and_dictionary':
            result = [checker.check_syllable(syllable) for syllable in

```

```

syllables]
    else:
        result = [syllable if syllable in checker.dictionary else
f"<{syllable}>" for syllable in syllables]
        formatted_result = ' '.join(result)
        results.append(formatted_result)

with open("syllable_checked_result.txt", 'w', encoding='utf-8') as f:
    f.write("\n".join(results))

with open("syllable_checked_result.txt", 'r', encoding='utf-8') as f:
    checked_lines = f.readlines()

normalized_lines = []
for checked_line in checked_lines:
    if '<' in checked_line:
        normalized_line = normalizer.normalize(checked_line.replace('<',
')).replace('>', ''))
        normalized_lines.append(normalized_line)
    else:
        normalized_lines.append(checked_line)

#if output_file:
    #with open(output_file, 'w', encoding='utf-8') as file:
        #file.write("\n".join(normalized_lines))
        #cleaned_normalized_lines = [line.strip() for line in
normalized_lines]
        #file.write("\n".join(cleaned_normalized_lines))

# Update 26 April 2026
if output_file:
    with open(output_file, 'w', encoding='utf-8') as file:
        for line in normalized_lines:
            # Ensure each line ends with exactly one newline
            file.write(line.strip() + "\n")

else:
    for output in normalized_lines:
        print(output.strip())

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Check Burmese syllable
structure and normalize text.")
    parser.add_argument('--input', '-i', help="Input file containing syllables,
or use stdin.")
    parser.add_argument('--output', '-o', help="Output file to save the
results.")
    parser.add_argument('--verbose', '-v', action='store_true', help="Verbose

```

```

mode: print line number, original syllable, and result.")
    parser.add_argument('--dictionary', '-d', help="Syllable dictionary
filename.")
    parser.add_argument('--frequency', '-f', type=int, default=2, help="Minimum
frequency to consider a syllable correct.")
    parser.add_argument('--check', '-c', choices=['dictionary',
'RE_and_dictionary'], default='dictionary', help="Control whether to check
syllables using the dictionary only or both regular expression rules and the
dictionary.")

    args = parser.parse_args()

    input_stream = open(args.input, 'r', encoding='utf-8') if args.input else
sys.stdin
    process_input(input_stream, args.output, args.verbose, args.dictionary,
args.frequency, args.check)

    if args.input:
        input_stream.close()

```

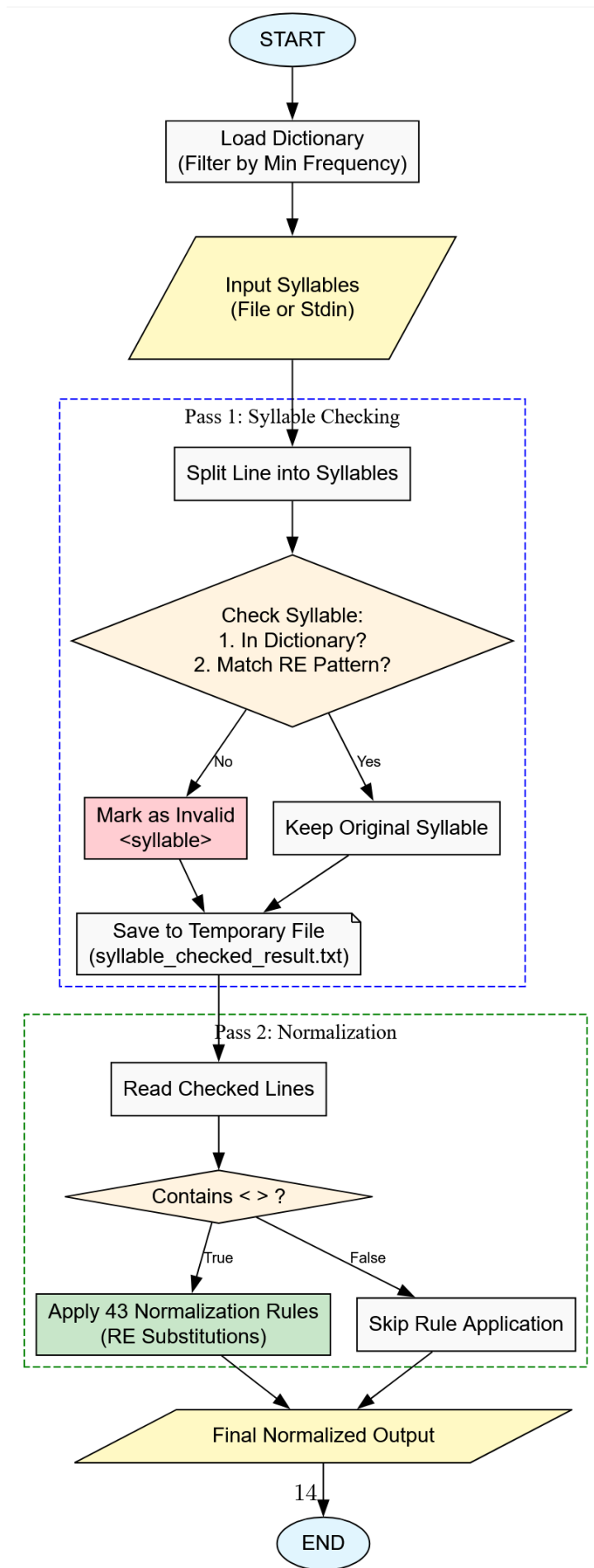
1.5.1 Algorithm of syl_normalizer

```

[36]: from IPython import display
display.Image("/home/ye/exp/SMT-NMT_tutorial/syl_normalizer/syl_normalizer.
↳png", width=300)

```

[36]:



အထက်က တုန်းက သုံးပြခဲ့တာက 2024 တုန်းက ဗားရှင်းပါ။ Notebook 2, 3 မှာက updated လုပ်ထားတဲ့ syl_normalization.py ကို သုံးထားပါတယ်။ GitHub မှာလည်း updated လုပ်ထားတဲ့ ဗားရှင်းကိုပဲ တင်ပေးထားလိုက်ပါမယ်။

Let's clean training, development and test dataset of Myanmar language

```
[57]: %cd /home/ye/exp/SMT-NMT_tutorial/syl_normalizer
```

```
/home/ye/exp/SMT-NMT_tutorial/syl_normalizer
```

```
[59]: !python ./syl_normalizer.py --help
```

```
usage: syl_normalizer.py [-h] [--input INPUT] [--output OUTPUT] [--verbose]
                        [--dictionary DICTIONARY] [--frequency FREQUENCY]
                        [--check {dictionary,RE_and_dictionary}]
```

Check Burmese syllable structure and normalize text.

options:

```
-h, --help            show this help message and exit
--input INPUT, -i INPUT
                        Input file containing syllables, or use stdin.
--output OUTPUT, -o OUTPUT
                        Output file to save the results.
--verbose, -v          Verbose mode: print line number, original syllable,
                        and result.
--dictionary DICTIONARY, -d DICTIONARY
                        Syllable dictionary filename.
--frequency FREQUENCY, -f FREQUENCY
                        Minimum frequency to consider a syllable correct.
--check {dictionary,RE_and_dictionary}, -c {dictionary,RE_and_dictionary}
                        Control whether to check syllables using the
                        dictionary only or both regular expression rules and
                        the dictionary.
```

```
[60]: !python ./syl_normalizer.py --dictionary ./final_syl_dictionary_13Feb2024.
      ↪sorted.txt
--frequency 2 --input ../clean-data/normalize/train.my \
--output ../clean-data/normalize/train.my.clean
```

```
[61]: !wc ../clean-data/normalize/train.my
```

```
62625  734742 7067941 ../clean-data/normalize/train.my
```

```
[62]: !wc ../clean-data/normalize/train.my.clean
```

```
62625  734667 7029308 ../clean-data/normalize/train.my.clean
```

```
[63]: !head ../clean-data/normalize/train.my.clean
```

အောက် ပါ ရော့ ဂါ လက္ခဏာ တွေ များ ရှိ ရင်
စိတ် ကို အေး အေး ဆေး ဆေး ထား ပါ
နာ တာ ရှည် ရော့ ဂါ ရယ် လို့ မ ရှိ ပါ ဘူး ဗျ
လက္ခဏာ တွေ နဲ့
ပြီး တဲ့ အ ခါ နား ကြပ် ကို တံ တော င် ဆစ် နား မှာ ထား မှာ မို့ ပါ ရှင်
ပု စွန် တော့ ကျွန် တော် စား တယ်
သို့ ပေ မဲ့ လည်း
ကျွန် တော် ခေါင်း ပေါ် ရေ ခဲ တင် ထား ရ မ လား
ကွန် ပြု တာ ရောင် ခြည် နဲ့ ထပ် စစ် ဆေး ပါ မယ် နော်
ဒီ ကြား မှာ ခ ဏ လေး ထိုင် စော င် ပေး လို့ ရ မ လား

```
[64]: !head ../clean-data/normalize/train.my
```

အောက် ပါ ရော့ ဂါ လက္ခဏာ တွေ များ ရှိ ရင်
စိတ် ကို အေး အေး ဆေး ဆေး ထား ပါ
နာ တာ ရှည် ရော့ ဂါ ရယ် လို့ မ ရှိ ပါ ဘူး ဗျ
လက္ခဏာ တွေ နဲ့
ပြီး တဲ့ အ ခါ နား ကြပ် ကို တံ တော င် ဆစ် နား မှာ ထား မှာ မို့ ပါ ရှင်
ပု စွန် တော့ ကျွန် တော် စား တယ်
သို့ ပေ မဲ့ လည်း
ကျွန် တော် ခေါင်း ပေါ် ရေ ခဲ တင် ထား ရ မ လား
ကွန် ပြု တာ ရောင် ခြည် နဲ့ ထပ် စစ် ဆေး ပါ မယ် နော်
ဒီ ကြား မှာ ခ ဏ လေး ထိုင် စော င် ပေး လို့ ရ မ လား

```
[65]: !tail ../clean-data/normalize/train.my.clean
```

စိတ် မ ပူ ပါ နဲ့ တော့ ရှင်
လက် နဲ့ ခြေ ထောက် တွေ မှာ လှုပ် ရှား မှု လျော့ နည်း သွား ပါ မယ်
အ သုံး ပြု တာ လျော့ ပါ
အ ပူ ရှိ တာ ကြော င် လည်း ဖြစ် တာ ပေါ့ ခင် ဗျာ
အဲ တာ က ယား တာ လား ဒါ ဆို နှင်း ခူ ဖြစ် တယ်
သွား ကျိုး ရှိ လား
သို့ မ ဟုတ် ဘယ် သူ တွေ များ လာ လို့ ရ သေး လဲ ခင် ဗျာ
နည်း နည်း တော့ စောင်း ကာ တိုက် ပေး ပါ
ဖိ နပ် ချွတ် ပြီး တာ နဲ့ ဒီ မှာ ရပ် ရ မယ် နော် ခင် ဗျာ
အ ခြား ရော့ ဂါ လက္ခဏာ မျိုး ရော့ ရှိ လား ရှ င်

```
[66]: !tail ../clean-data/normalize/train.my
```

စိတ် မ ပူ ပါ နဲ့ တော့ ရှင်
လက် နဲ့ ခြေ ထောက် တွေ မှာ လှုပ် ရှား မှု လျော့ နည်း သွား ပါ မယ်
အ သုံး ပြု တာ လျော့ ပါ

အ ပူ ရှိ တာ ကြော င် လည်း ဖြစ် တာ ပေါ့ ခင် ဗျာ
 အဲ တာ က ယား တာ လား ဒါ ဆို နှင်း ခူ ဖြစ် တယ်
 သွား ကျိုး ရှိ လား
 သို့ မ ဟုတ် ဘယ် သူ တွေ များ လာ လို့ ရ သေး လဲ ခင် ဗျာ
 နည်း နည်း တော့ စောင်း ကာ တိုက် ပေး ပါ
 ဖိ နပ် ချွတ် ပြီး တာ နဲ့ ဒီ မှာ ရပ် ရ မယ် နော် ခင် ဗျာ
 အ ခြား ရော ဂါ လက္ခဏာ မျိုး ရော ရှိ လား ရှ င်

Development data cleaning ...

```
[67]: !python ./syl_normalizer.py --dictionary ./final_syl_dictionary_13Feb2024.
      ↪sorted.txt
      --frequency 2 --input ../clean-data/normalize/dev.my \
      --output ../clean-data/normalize/dev.my.clean
```

```
[68]: !wc ../clean-data/normalize/dev.my

10000 117884 1133121 ../clean-data/normalize/dev.my
```

```
[70]: !wc ../clean-data/normalize/dev.my.clean

10000 117871 1126953 ../clean-data/normalize/dev.my.clean
```

Test data cleaning ...

```
[71]: !python ./syl_normalizer.py --dictionary ./final_syl_dictionary_13Feb2024.
      ↪sorted.txt
      --frequency 2 --input ../clean-data/normalize/test.my \
      --output ../clean-data/normalize/test.my.clean
```

```
[72]: !wc ../clean-data/normalize/test.my

8000 93919 901455 ../clean-data/normalize/test.my
```

```
[73]: !wc ../clean-data/normalize/test.my.clean

8000 93908 896405 ../clean-data/normalize/test.my.clean
```

```
[141]: %cd /home/ye/exp/SMT-NMT_tutorial/clean-data/normalize/

/home/ye/exp/SMT-NMT_tutorial/clean-data/normalize
```

```
[143]: !head ./test.my
```

ခွဲ မှ ရ မှာ လား
 မျက် ခံ လှုပ် တဲ့ လက္ခဏာ ရှိ မယ်
 အ ရက် ကို စ နဲ့ လွှတ် ရ မည် ဖြစ် ပြီး ပ ရှိ တင်း နှ င် ဗီ တာ မင် ဓာတ် ကြွယ် ဝ
 သော အာ ဟာ ရ ရှိ သော အ စား အ စာ များ ကို စား သ င် သည်
 ဟုတ် ကဲ့ သူ ဖျား ပါ တယ် သူ သီး သန့် တော့ မ နေ ခဲ့ ဘူး
 မောင် လေး နောက် ဆုံး အ ကြိမ် ဆေး သောက် ခဲ့ တာ ဘယ် ချိန် လဲ

ကိုယ် မှာ ကူး စက် တာ က နေ ဦး နောက် ထိ ပါ ကူး စက် လာ တယ်
 မစ် စ တာ ပ ရာ မို့ ရဲ့ ဆွေ မျိုး ဝင် လာ ဖို့ တစ် ချက် ခေါ် ပေး ပါ ရှင်
 အ ချို့ ကို လျှော့ ချ ကြ ည့် လိုက် ပါ
 သူ တို့ နဲ့ အ တူ တူ ရှိ နေ တယ် လို့ ပဲ တွေး မိ နေ တယ် ဗျာ
 ဆေး ယူ ပြီး ပြီ ဆို ရင် တော့ အိမ် ပြန် လို့ ရ ပြီ နော်

```
[144]: !head ./test.my.clean
```

ခွဲ မှ ရ မှာ လား
 မျက် ခံ လှုပ် တဲ့ လက္ခဏာ ရှိ မယ်
 အ ရက် ကို စွ န် လွှတ် ရ မည် ဖြစ် ပြီး ပ ရှိ တင်း နှ င် ဗီ တာ မင် ဓာတ် ကြွယ် ဝ
 သော အာ ဟာ ရ ရှိ သော အ စား အ စာ များ ကို စား သ င် သည်
 ဟုတ် ကဲ့ သူ ဖျား ပါ တယ် သူ သီး သန့် တော့ မ နေ ခဲ့ ဘူး
 မောင် လေး နောက် ဆုံး အ ကြိမ် ဆေး သောက် ခဲ့ တာ ဘယ် ချိန် လဲ
 ကိုယ် မှာ ကူး စက် တာ က နေ ဦး နောက် ထိ ပါ ကူး စက် လာ တယ်
 မစ် စ တာ ပ ရာ မို့ ရဲ့ ဆွေ မျိုး ဝင် လာ ဖို့ တစ် ချက် ခေါ် ပေး ပါ ရှင်
 အ ချို့ ကို လျှော့ ချ ကြ ည့် လိုက် ပါ
 သူ တို့ နဲ့ အ တူ တူ ရှိ နေ တယ် လို့ ပဲ တွေး မိ နေ တယ် ဗျာ
 ဆေး ယူ ပြီး ပြီ ဆို ရင် တော့ အိမ် ပြန် လို့ ရ ပြီ နော်

cleaning လုပ်ထားတဲ့ ဖိုင်တွေကို machine translation လုပ်တဲ့အခါမှာ သုံးဖို့အတွက် ကော်ပီကူး
 ပြင်ထားကြရအောင်

```
[75]: %pwd
```

```
[75]: '/home/ye/exp/SMT-NMT_tutorial/syl_normalizer'
```

```
[76]: %cd ../clean-data/
```

```
/home/ye/exp/SMT-NMT_tutorial/clean-data
```

```
[77]: !cp ./normalize/*.clean .
```

```
[79]: !tree .
```

```
.
├── dev.my.clean
├── normalize
│   ├── dev.my
│   ├── dev.my.clean
│   ├── dev.th
│   ├── test.my
│   ├── test.my.clean
│   ├── test.th
│   ├── train.my
│   ├── train.my.clean
│   └── train.th
```

```
test.my.clean
train.my.clean
```

2 directories, 12 files

.clean တွေကို ဖြုတ်ကြရအောင်

SMT မှာက နောက်ဆုံး ဖိုင်နာမည်နောက်ဆုံးပိုင်း (i.e. file extension) က source/target ဘာသာစကားတွေအဖြစ် ယူဆပြီး အလုပ်လုပ်မှာမို့လို့။

```
[80]: !mv train.my.clean train.my
```

```
[81]: !mv dev.my.clean dev.my
```

```
[82]: !mv test.my.clean test.my
```

```
[83]: !tree .
```

```
.
├── dev.my
├── normalize
│   ├── dev.my
│   ├── dev.my.clean
│   ├── dev.th
│   ├── test.my
│   ├── test.my.clean
│   ├── test.th
│   ├── train.my
│   ├── train.my.clean
│   └── train.th
├── test.my
└── train.my
```

2 directories, 12 files

1.6 Moses SMT Framework

အောက်ပါ လင့်တွေကနေ လေ့လာပါ။ ထဲထဲဝင်ဝင် သိချင်ရင်တော့ user manual မှာ လက်တွေ့ ဘယ်လို run ရမယ် ဆိုတာ အသေးစိတ် ရေးထားတာမို့ ဖတ်ရှုလေ့လာကြပါ။ SMT ထဲက မော်ဒယ်တွေ၊ လက်တွေ့လုပ်ပုံတွေ နားလည်ရင် အလုပ်ထဲမှာ ဖြေရှင်းကြရမယ့် အင်ဂျင်နီယာကိစ္စတွေအတွက် အများကြီး အသုံးဝင်ပါလိမ့်မယ်။

1. homepage: <https://www2.statmt.org/moses/>
2. User Manual: <http://www2.statmt.org/moses/manual/manual.pdf>

ဆရာက moses framework ရဲ့ executable binary ဖိုင်တွေနဲ့ တခြား လိုအပ်တဲ့ script တွေကို အောက်ပါ path မှာ ပြင်ဆင်ထားပါတယ်။

```
[84]: !ls /home/ye/tool/mosesbin
```

```
ubuntu-17.04  ubuntu-17.04.tgz
```

```
[85]: %cd /home/ye/tool/mosesbin
```

```
/home/ye/tool/mosesbin
```

moses framework အထဲမှာ ပါဝင်တဲ့ ဖိုင်တွေနဲ့ သူ့ရဲ့ folder structure ကို အကြမ်းဖျဉ်း လေ့လာကြရအောင်။

```
[86]: %cd ubuntu-17.04/
```

```
/home/ye/tool/mosesbin/ubuntu-17.04
```

```
[87]: !tree .
```

```
.
├── moses
│   └── bin
│       ├── 1-1-Extraction
│       ├── addLexR0toPT
│       ├── biconcor
│       ├── build_binary
│       ├── config.log
│       ├── consolidate
│       ├── consolidate-direct
│       ├── consolidate-reverse
│       ├── CreateOnDiskPt
│       ├── CreateProbingPT
│       ├── CreateProbingPT2 -> CreateProbingPT
│       ├── dump_counts
│       ├── evaluator
│       ├── extract
│       ├── extract-ghkm
│       ├── extract-lex
│       ├── extract-mixed-syntax
│       ├── extractor
│       ├── extract-rules
│       ├── filter
│       ├── filter-rule-table
│       ├── fragment
│       ├── gcc-6.3.0
│       ├── debug
│       │   ├── empty_test_shared
│       │   ├── empty_test_shared.o
│       │   ├── empty_test_static
│       │   └── empty_test_static.o
│       ├── generateSequences
│       ├── hgdecode
│       ├── kbmira
│       ├── kenlm_benchmark
│       ├── lexical-reordering-score
│       └── lmbrgrid
```

```

lmplz
merge-sorted
mert
mmlex-build
moses
moses2
moses_chart -> moses
mosesserver
mtt-build
pcfg-extract
pcfg-score
phrase-lookup
phrase_table_vocab
postprocess-egret-forests
prepare-expected-bleu-training
pro
processLexicalTable
processLexicalTableMin
processPhraseTableMin
project-cache.jam
pruneGeneration
prunePhraseTable
query
queryLexicalTable
queryOnDiskPt
queryPhraseTableMin
relax-parse
score
score-stsg
sentence-bleu
sentence-bleu-nbest
statistics
symal
symal2mam
TMining
train-expected-bleu
vwtrainer
scripts
  analysis
    bootstrap-hypothesis-difference-significance.pl
    extract-target-trees.py
    nontranslated_words.pl
    oov.pl
    perllib
    Error.pm
    README
    sentence-by-sentence.pl
    sg2dot.perl

```

```

show-phrases-used.pl
smtgui
    Corpus.pm
    file-descriptions
    file-factors
    filter-phrase-table.pl
    newsmtgui.cgi
    README
    suspicious_tokenization.pl
    weight-scan.pl
    weight-scan-summarize.sh
docker
    Dockerfile.ubuntu.basic
    Dockerfile.ubuntu.fastlightpbmt
ems
    example
        config.basic
        config.factored
        config.hierarchical
        config.syntax
        config.toy
        config.toy.bilinguallm
    data
        config.hierarchical
        nc-5k.en
        nc-5k.fr
        run.hierarchical.sh
        test-ref.en.sgm
        test-src.fr.sgm
        weight_bilinguallm.ini
        weight.ini
    experiment.machines
    experiment.meta
    experiment.perl
    fix-info.perl
    support
        analysis.perl
        berkeley-process.sh
        berkeley-train.sh
        build-domain-file-from-subcorpora.perl
        build-sparse-features.perl
        cache-model.perl
        consolidate-training-data.perl
        create-xml.perl
        defaultconfig.py
        fast-align-in-parts.perl
        generic-multicore-parallelizer.perl
        generic-parallelizer.perl

```

input-from-sgm.perl
interpolate-lm.perl
lmplz-wrapper.perl
mml-filter.perl
mml-filter.py
mml-score.perl
mml-train.perl
prepare-fast-align.perl
reference-from-sgm.perl
remove-segmentation-markup.perl
report-experiment-scores.perl
run-command-on-multiple-refsets.perl
run-wade.perl
split-sentences.perl
submit-grid.perl
substitute-filtered-tables-and-weights.perl
substitute-filtered-tables.perl
substitute-weights.perl
symmetrize-fast-align.perl
ter.perl
thot-lm-wrapper.perl
tree-converter-wrapper.perl
wrap-xml.perl

web

analysis_diff.php
analysis.php
base64.js
bilingual-concordance.css
close.gif
comment.php
diff.php
favicon.ico
general.css
hierarchical-segmentation.css
hierarchical-segmentation.js
index.php

javascripts

builder.js
controls.js
dragdrop.js
effects.js
prototype.js
scriptaculous.js
scriptaculous-js-1.8.3

CHANGELOG

lib

prototype.js
MIT-LICENSE

README.rdoc

src

- builder.js
- controls.js
- dragdrop.js
- effects.js
- scriptaculous.js
- slider.js
- sound.js
- unittest.js

test

functional

- ajax_autocompleter2_test.html
- ajax_autocompleter_test.html
- ajax_inplacecollectioneditor_test.html
- _ajax_inplaceeditor_result.html
- ajax_inplaceeditor_test.html
- _ajax_inplaceeditor_text.html
- _autocomplete_result.html
- _autocomplete_result_single.html
- dragdrop2_test.html
- dragdrop3_test.html
- dragdrop4_test.html
- dragdrop5_test.html
- dragdrop6_test.html
- dragdrop7_test.html
- dragdrop8_test.html
- dragdrop9_test.html
- dragdrop_delay_test.html
- dragdrop_test.html
- dropmarker.png
- effect_direct_test.html
- effect_puff_test.html
- effects2_test.html
- effects3_test.html
- effects4_test.html
- effects5b_test.html
- effects5_test.html
- effects6_test.html
- effects_blind_test.html
- effect_scale_test.html
- effects_float_appear_test.html
- effects_grow_strink_test.html
- effect_shake.html
- effects_highlight_bg_image.html
- effects_queue_limit_test.html
- effects_queue_test.html
- effects_random_demo.html

```

effects_test.html
effects_toggle_test.html
icon.png
index.html
position_clone_test.html
slider_test.html
sortable2_test.html
sortable3_test.html
sortable4_test.html
sortable5_test.html
sortable6_test.html
sortable_nested_test.html
sortable_test.html
sortable_tree_test.html
sound_test.html
sword.mp3
texteffects_test.html
run_functional_tests.html
run_unit_tests.html
test.css
unit
    ajax_autocompleter_test.html
    _ajax_inplaceeditor_ipce_alt_text.html
    _ajax_inplaceeditor_result2.html
    _ajax_inplaceeditor_result.html
    _ajax_inplaceeditor_tagged.html
    ajax_inplaceeditor_test.html
    _ajax_inplaceeditor_text.html
    _ajax_updater_result.html
    _autocomplete_result.html
    _autocomplete_result_nobr.html
    bdd_test.html
    builder_test.html
    dragdrop_test.html
    effects_test.html
    element_test.html
    icon.png
    index.html
    loading_test.html
    position_clone_test.html
    slider_test.html
    sortable_test.html
    string_test.html
    unittest_test.html
slider.js
sound.js
unittest.js
lib.php

```

- overview.php
 - progress.perl
 - setup
 - sgviz.js
 - sgviz.php
 - spinner.gif
- fuzzy-match
 - compile.sh
 - create_xml.perl
- generic
 - binarize4moses2.perl
 - bsbleu.py
 - compound-splitter.perl
 - extract-factors.pl
 - extract-parallel.perl
 - fsa2fsal.pl
 - fsa2plf.pl
 - fsal2fsa.pl
 - fsa-sample.fsa
 - generic-parallel.perl
 - giza-parallel.perl
 - lopar2pos.pl
 - moses-parallel.pl
 - moses_sim_pe.py
 - mteval-v11b.pl
 - mteval-v12.pl
 - mteval-v13a.pl
 - mteval-v14.pl
 - multi-bleu.perl
 - multi_moses.py
 - ph_numbers.perl
 - qsub-wrapper.pl
 - reverse-alignment.perl
 - score-parallel.perl
 - score_parallel.py
 - strip-xml.perl
 - trainlm-irst2.perl
- Jamfile
- nbest-rescore
 - README.md
 - rescore.py
 - topbest.py
 - train.py
- OSM
 - extract-singletons.perl
 - flipAlignment.perl
 - OSM-Train.perl
- other

```

beautify.py
blame-stat.sh
buckwalter.perl
convert-pt.perl
delete-scores.perl
gacha_filter.py
get_many_translations_from_google.perl
retain-lines.perl
translate_by_microsoft_bing.perl
README
recaser
    dettruecase.perl
    recase.perl
    train-recaser.perl
    train-truecaser.perl
    truecase.perl
regression-testing
    compare-results.pl
    create_localized_moses_ini.pl
    modify-pars.pl
    MosesScriptsRegressionTesting.pm
    moses-scripts-reg-test-data-1.0
        lm
            train.en.w5.msb.nops.blm
        models
            phrase-table.0-0.gz
            reordering-table.msd-bidirectional-
fe.0.5.0-0.gz
    moses-virtual
    moses-virtual.pl
    run-single-test.pl
    run-test-suite.pl
    tests
        mert-moses-new
            command
            data
                archive.list
                run1.NBEST.out.gz
                run1.TRANS.out
                run2.NBEST.out.gz
                run2.TRANS.out
                run3.NBEST.out.gz
                run3.TRANS.out
                run4.NBEST.out.gz
                run4.TRANS.out
                run5.NBEST.out.gz
                run5.TRANS.out

```

```

        run6.NBEST.out.gz
        run6.TRANS.out
        run7.NBEST.out.gz
        run7.TRANS.out
filter-stderr
filter-stdout
input
moses.ini
reference.0
reference.1
reference.2
truth
        results.dat
mert-moses-new-aggregate
command
data
        archive.list
        run10.NBEST.out.gz
        run10.TRANS.out
        run11.NBEST.out.gz
        run11.TRANS.out
        run12.NBEST.out.gz
        run12.TRANS.out
        run13.NBEST.out.gz
        run13.TRANS.out
        run14.NBEST.out.gz
        run14.TRANS.out
        run1.NBEST.out.gz
        run1.TRANS.out
        run2.NBEST.out.gz
        run2.TRANS.out
        run3.NBEST.out.gz
        run3.TRANS.out
        run4.NBEST.out.gz
        run4.TRANS.out
        run5.NBEST.out.gz
        run5.TRANS.out
        run6.NBEST.out.gz
        run6.TRANS.out
        run7.NBEST.out.gz
        run7.TRANS.out
        run8.NBEST.out.gz
        run8.TRANS.out
        run9.NBEST.out.gz
        run9.TRANS.out
filter-stderr
filter-stdout
input

```

```

moses.ini
reference.0
reference.1
reference.2
truth
    results.dat
mert-moses-new-continue
command
data
    actual.index
    archive.list
    finished_step.txt
    run1.features.dat
    run1.scores.dat
    run2.features.dat
    run2.scores.dat
    run3.features.dat
    run3.scores.dat
    run4.features.dat
    run4.scores.dat
    run5.features.dat
    run5.mert.log
    run5.names.txt
    run5.scores.dat
    run5.weights.txt
    run6.NBEST.out.gz
    run6.TRANS.out
    run7.NBEST.out.gz
    run7.TRANS.out
filter-stderr
filter-stdout
input
moses.ini
reference.0
reference.1
reference.2
truth
    results.dat
mert-moses-new-nocase
command
data
    archive.list
    run1.NBEST.out.gz
    run1.TRANS.out
    run2.NBEST.out.gz
    run2.TRANS.out
    run3.NBEST.out.gz
    run3.TRANS.out

```

```
run4.NBEST.out.gz
run4.TRANS.out
run5.NBEST.out.gz
run5.TRANS.out
filter-stderr
filter-stdout
input
moses.ini
reference.0
reference.1
reference.2
truth
results.dat

server
  moses.py
  sim-pe.py
share
  nonbreaking_prefixes
    nonbreaking_prefix.ca
    nonbreaking_prefix.cs
    nonbreaking_prefix.de
    nonbreaking_prefix.el
    nonbreaking_prefix.en
    nonbreaking_prefix.es
    nonbreaking_prefix.fi
    nonbreaking_prefix.fr
    nonbreaking_prefix.ga
    nonbreaking_prefix.hu
    nonbreaking_prefix.is
    nonbreaking_prefix.it
    nonbreaking_prefix.lt
    nonbreaking_prefix.lv
    nonbreaking_prefix.nl
    nonbreaking_prefix.pl
    nonbreaking_prefix.pt
    nonbreaking_prefix.ro
    nonbreaking_prefix.ru
    nonbreaking_prefix.sk
    nonbreaking_prefix.sl
    nonbreaking_prefix.sv
    nonbreaking_prefix.ta
    nonbreaking_prefix.yue
    nonbreaking_prefix.zh
    README.txt

tests
  cs-en-sample
    lm.en.gz
    test.ref
```

```

    test.src
    train.cs
    train.en
full-train-mert-decode.test
README
train-factored-test-step9.test
tokenizer
    basic-protected-patterns
    deescape-special-chars.perl
    deescape-special-chars-PTB.perl
    delete-long-words.perl
    detokenizer.perl
    escape-special-chars.perl
    lowercase.perl
    normalize-punctuation.perl
    pre-tok-clean.perl
    pre_tokenize_cleaning.py
    pre-tokenizer.perl
    remove-non-printing-char.perl
    replace-unicode-punctuation.perl
    tokenizer.perl
    tokenizer_PTB.perl
training
    absolutize_moses_model.pl
    analyse_moses_model.pl
bilingual-lm
    averageNullEmbedding.py
    create_blm_ini.py
    extract.py
    extract_test.py
    extract_training.py
    README
    reduce_ngrams.py
    tag.sh
    test_nplm.py
    train_nplm.py
    binarize-model.perl
    build-generation-table.perl
    build-mmsapt.perl
    clean-corpus-n.perl
    clean-corpus-n-ratio.perl -> clean-
corpus-n.perl
    clone_moses_model.pl
    combine_factors.pl
    combine_factors_syntax.pl
    convert-moses-ini-to-v2.perl
    corpus-sizes.perl

```

```

create_count_tables.py
create_nplm_ini.py
exodus.perl
filter-model-given-input.pl
filter-rule-table.py
flexibility_score.py
get-lexical.perl
giza2bal.pl
Jamfile
LexicalTranslationModel.pm
mert-moses.pl
postprocess-lopap.perl
rdlm
    average_null_embedding.py
    extract_syntactic_ngrams.py
    extract_vocab.py
    README
    train_rdlm.py
reduce_combine.pl
reduce-factors.perl
reduce-topt-count.pl
remove-orphan-phrase-pairs-from-reordering-
table.perl

strip-xml.perl
threshold-filter.perl
train-global-lexicon-model.perl
train-model.perl
train-neurallm.py
wrap_moses.py
wrappers
    adam-suffix-array
        suffix-array-create.sh
        suffix-array-extract.sh
    berkeleyparsed2mosesxml.perl
    berkeleyparsed2mosesxml_PTB.perl
    conll2mosesxml.py
    filter-excluded-lines.perl
    find-unparseable.perl
    madamira-tok.perl
    madamira-wrapper.perl
    mada-wrapper.perl
    make-factor-brown-cluster-mkcls.perl
    make-factor-de-lemma.perl
    make-factor-de-morph.perl
    make-factor-de-pos.perl
    make-factor-en-porter.perl
    make-factor-en-pos.mxpost.perl

```

```

    make-factor-pos.tree-tagger.perl
    make-factor-stem.perl
    make-factor-suffix.perl
    morfessor-wrapper.perl
    mosesxml2berkeleyparsed.perl
    mosesxml2brackets.py
    parse-de-berkeley.perl
    parse-de-bitpar.perl
    parse-en-bllip.perl
    parse-en-collins.perl
    parse-en-egret.perl
    parse-en-senna.perl
    parse-en-stanford.py
    senna2brackets.py
    syntax-hyphen-splitting.perl
    tagger-german-chunk.perl
    tree-converter-mosesxml.sh
Transliteration
    clean.pl
    corpusCreator.pl
    in-decoding-transliteration.pl
    post-decoding-transliteration.pl
    prepare-transliteration-phrase-table.pl
    threshold.pl
    train-transliteration-module.pl
training-tools
    merge_alignment.py
    mgiza
    mkcls
    snt2cooc

```

58 directories, 579 files

1.7 Phrase Based SMT Tutorial

YTU မှာ ဆရာ workshop လုပ်တုန်းက တင်ပေးထားခဲ့တဲ့ configuration ဖိုင်စတာတွေက အသုံးဝင်ပါလိမ့်မယ်။

MTRSS: Machine Translation Research Summer School

Here, I uploaded some files that I used at MTRSS (8-10 Oct, 2019), Yangon Technological University (YTU), Yangon, Myanmar.

Link: <https://github.com/ye-kyaw-thu/MTRSS/tree/master/pbsmt>

ဒီကနေ စပြီး လက်တွေ့ PBSMT ကို run ဖို့ စပြင်ပြီး step-by-step လုပ်ပြသွားမယ်။

```
[88]: %pwd
```

```
[88]: '/home/ye/tool/mosesbin/ubuntu-17.04'
```

1.8 Preparing Testset SGM Files

ခု လက်တွေ့လုပ်ငြမယ့် SMT က moses2 ရဲ့ experimental system ပုံစံမျိုး configuration file ဆောက်ပြီးသွားမှာမို့လို့ SGM ဖိုင်ကို ရှိမရှိ စစ်တယ်။ မတွေ့ရင် run မလုပ်ပေးဘူး။ အဲဒီအတွက် test data တွေကို SGM file format အရင် ပြောင်းကြရအောင်။

ဆရာပြင်ပေးထားတဲ့ perl script သုံးဖိုင်က အောက်ပါ path အောက်မှာရှိတယ်။ ဒီ Notebook နဲ့ အတူလည်း ရှိပေးထားပါမယ်။

<https://github.com/ye-kyaw-thu/MTRSS/tree/master/pbsmt/data/test-sgm>

```
[149]: %cd /home/ye/exp/SMT-NMT_tutorial/clean-data/scripts/
```

```
/home/ye/exp/SMT-NMT_tutorial/clean-data/scripts
```

```
[150]: !cat generate_sgms.pl
```

```
#!/usr/bin/perl
use strict;

# written by Ye, NECTEC
# for MTRSS, YTU, Myanmar

my @langs;

foreach my $trainFile ( </home/ros/experiment/my-rk/data/train.[a-z][a-z]> )
foreach my $trainFile ( <../train.[a-z][a-z]> )
{
    $trainFile =~ m/train.([a-z][a-z])/;
    push @langs, $1;
}

foreach my $lang (@langs)
{
    `perl ./ref2sgm.pl $lang > test.$lang.ref.sgm`;
    `perl ./src2sgm.pl $lang > test.$lang.src.sgm`;
}
```

အထက်က perl script က ပြင်စရာ မလိုပါ။ အောက်က နောက်ထပ် perl script နှစ်ဖိုင်ကိုတော့ ကိုယ်လုပ်မယ့် MT experiment နဲ့ ဆိုင်တဲ့ information ကို ဝင်ရေးပါ။

```
[151]: !cat ./src2sgm.pl
```

```
#!/usr/bin/perl
use strict;

# written by Ye, NECTEC
# for MTRSS, YTU, Myanmar

my $src = shift;
```

```

print "<srcset setid=\"Thai-Myanmar_data\" srclang=\"any\">\n";
print "<doc docid=\"none\" genre=\"8000\" origlang=\"$src\">\n";

#open FILE, "/home/ros/experiment/my-rk/data/test.$src" or die;
open FILE, "../test.$src" or die;

my $id=1;

while( <FILE> )
{
    chomp;

    print "<seg id=\"$id\">$_ </seg>\n";
    $id++;
}

print "</doc>\n</srcset>\n";
နောက်ထပ် တစ်ပိုင်းက reference အတွက်ပါ။

```

[152]: !cat ./ref2sgm.pl

```

#!/usr/bin/perl
use strict;

# written by Ye, NECTEC
# for MTRSS, YTU, Myanmar

my $trg = shift;

#print "<refset trglang=\"$trg\" setid=\"sl_data\" srclang=\"any\">\n";
print "<refset trglang=\"$trg\" setid=\"Thai-Myanmar_data\" srclang=\"any\">\n";
print "<doc sysid=\"ref\" docid=\"none\" genre=\"8000\" origlang=\"any\">\n";

#open FILE, "/home/ros/experiment/my-rk/data/test.$trg" or die;
open FILE, "../test.$trg" or die;

my $id=1;

while( <FILE> )
{
    chomp;

    print "<seg id=\"$id\">$_ </seg>\n";
    $id++;
}

```

```
print "</doc>\n</refset>\n";
```

1.8.1 Run perl script

generate လုပ်ပေးမယ့် perl ကို run လိုက်ရင်အဲဒီ ဖိုလ်ဒါ path အောက်မှာပဲ SGM ဖိုင်တွေ ထွက်လာလိမ့်မယ်။

```
[153]: !perl ./generate_sgms.pl
```

```
[154]: !ls *sgm
```

```
test.my.ref.sgm test.my.src.sgm test.th.ref.sgm test.th.src.sgm
```

```
[155]: !head test.my.ref.sgm
```

```
<refset trglang="my" setid="Thai-Myanmar_data" srclang="any">
<doc sysid="ref" docid="none" genre="8000" origlang="any">
<seg id="1">ခွဲ မှ ရ မှာ လား </seg>
<seg id="2">မျက် ခံ လှုပ် တဲ့ လက္ခဏာ ရှိ မယ် </seg>
<seg id="3">အ ရက် ကို စွ န် လွတ် ရ မည် ဖြစ် ပြီး ပ ရှိ တင်း နှ င် ဗီ တာ မင်
ဓာတ် ကြွယ် ဝ သော အာ ဟာ ရ ရှိ သော အ စား အ စာ များ ကို စား သ င် သည် </
<seg>
<seg id="4">ဟုတ် ကဲ့ သူ ဖျား ပါ တယ် သူ သီး သန့် တော့ မ နေ ခဲ့ ဘူး </seg>
<seg id="5">မောင် လေး နောက် ဆုံး အ ကြိမ် ဆေး သောက် ခဲ့ တာ ဘယ် ချိန် လဲ </seg>
<seg id="6">ကိုယ် မှာ ကူး စက် တာ က နေ ဦး နောက် ထိ ပါ ကူး စက် လာ တယ် </
<seg>
<seg id="7">မစ် စ တာ ပ ရာ မို့ ရဲ့ ဆွေ မျိုး ဝင် လာ ဖို့ တစ် ချက် ခေါ် ပေး ပါ
ရှင် </seg>
<seg id="8">အ ချို့ ကို လျော့ ချ ကြ ည့် လိုက် ပါ </seg>
```

```
[156]: !head test.th.ref.sgm
```

```
<refset trglang="th" setid="Thai-Myanmar_data" srclang="any">
<doc sysid="ref" docid="none" genre="8000" origlang="any">
<seg id="1"> </seg>
<seg id="2"> </seg>
<seg id="3">
</seg>
<seg id="4"> </seg>
<seg id="5"> </seg>
<seg id="6"> </seg>
<seg id="7"> </seg>
<seg id="8"> </seg>
```

```
[157]: %cd /home/ye/exp/SMT-NMT_tutorial
```

```
/home/ye/exp/SMT-NMT_tutorial
```

1.9 GIZA++ Installation

Alignment လုပ်ပေးမယ့် tool က ကိုယ့်စက်ထဲမှာ မရှိသေးရင် git clone လုပ်ပြီး install လုပ်ပါ။ ပြီးရင် အဲဒီ path ကို config မှိန်မှာ ဝင်ဖြည့်ပေးပါ။

git clone <https://github.com/moses-smt/giza-pp> လုပ်ပြီး အဲဒီ path အောက်ကို ဝင်ပြီး make all command ကို run ပါ။

```
ye@lst-hpc3090:~/tool/giza-pp$ make all ... .. g++ -O3 -W -Wall plain2snt.cpp -o plain2snt.out g++ -O3 -g -W -Wall snt2cooc.cpp -o snt2cooc.out make[1]: Leaving directory '/home/ye/tool/giza-pp/GIZA++-v2' make -C mkcls-v2 make[1]: Entering directory '/home/ye/tool/giza-pp/mkcls-v2' g++ -Wall -W -DNDEBUG -O3 -funroll-loops -c GDAOptimization.cpp -o GDAOptimization.o g++ -Wall -W -DNDEBUG -O3 -funroll-loops -c HCOptimization.cpp -o HCOptimization.o g++ -Wall -W -DNDEBUG -O3 -funroll-loops -c Problem.cpp -o Problem.o g++ -Wall -W -DNDEBUG -O3 -funroll-loops -c IterOptimization.cpp -o IterOptimization.o g++ -Wall -W -DNDEBUG -O3 -funroll-loops -c ProblemTest.cpp -o ProblemTest.o g++ -Wall -W -DNDEBUG -O3 -funroll-loops -c RRTOptimization.cpp -o RRTOptimization.o g++ -Wall -W -DNDEBUG -O3 -funroll-loops -c MYOptimization.cpp -o MYOptimization.o g++ -Wall -W -DNDEBUG -O3 -funroll-loops -c SAOptimization.cpp -o SAOptimization.o g++ -Wall -W -DNDEBUG -O3 -funroll-loops -c TAOptimization.cpp -o TAOptimization.o g++ -Wall -W -DNDEBUG -O3 -funroll-loops -c Optimization.cpp -o Optimization.o g++ -Wall -W -DNDEBUG -O3 -funroll-loops -c KategProblemTest.cpp -o KategProblemTest.o g++ -Wall -W -DNDEBUG -O3 -funroll-loops -c KategProblemKBC.cpp -o KategProblemKBC.o g++ -Wall -W -DNDEBUG -O3 -funroll-loops -c KategProblemWBC.cpp -o KategProblemWBC.o g++ -Wall -W -DNDEBUG -O3 -funroll-loops -c KategProblem.cpp -o KategProblem.o g++ -Wall -W -DNDEBUG -O3 -funroll-loops -c StatVar.cpp -o StatVar.o g++ -Wall -W -DNDEBUG -O3 -funroll-loops -c general.cpp -o general.o g++ -Wall -W -DNDEBUG -O3 -funroll-loops -c mkcls.cpp -o mkcls.o g++ -Wall -W -DNDEBUG -O3 -funroll-loops -o mkcls GDAOptimization.o HCOptimization.o Problem.o IterOptimization.o ProblemTest.o RRTOptimization.o MYOptimization.o SAOptimization.o TAOptimization.o Optimization.o KategProblemTest.o KategProblemKBC.o KategProblemWBC.o KategProblem.o StatVar.o general.o mkcls.o make[1]: Leaving directory '/home/ye/tool/giza-pp/mkcls-v2' ye@lst-hpc3090:~/tool/giza-pp$
```

PBSMT ကို ဖိုလ်ဒါ အသစ်တစ်ခုဆောက်ပြီး အဲဒီအောက်မှာပဲ run သွားမယ်။

```
[90]: !mkdir pbsmt
```

```
[160]: %cd /home/ye/exp/SMT-NMT_tutorial/pbsmt
```

```
/home/ye/exp/SMT-NMT_tutorial/pbsmt
```

kc-my.config.baseline ကို လေ့လာရင်း အဲဒီဖိုလ်ဒါကို ထိုင်း-မြန်မာ ဘာသာပြန်ဖို့အတွက် ဝင်ပြင်ကြရအောင်။

```
[106]: !cat kc-my.config.baseline
```

```
### directories that contain tools and data
#
# moses
#moses-src-dir = /home/ros/mosesdecoder
moses-src-dir = /home/lar/tool/moses/
```

```

# moses binaries
moses-bin-dir = $moses-src-dir/bin

# moses scripts
moses-script-dir = $moses-src-dir/scripts

# directory where GIZA++/MGIZA programs resides
external-bin-dir = /home/lar/tool/giza-pp-master/mkcls-v2
#external-bin-dir = /home/lar/tool/giza-pp-master/GIZA++-v2
#external-bin-dir = /home/lar/tool/giza-pp-master/
#external-bin-dir = /home/ye/tool/mgiza/mgizapp/bin

# srilm
#srilm-dir = /data/lttools/training/srilm/srilm-1.6.0/bin/i686-m64
#I haven't installed SRILM yet on Deep Learning Box computer
#srilm-dir = /home/ros/tool/srilm-1.7.1/bin/i686-m64

# firstlm
#firstlm-dir = $moses-src-dir/firstlm/bin

# randlm
#randlm-dir = $moses-src-dir/randlm/bin

# kenlm
lmplz = $moses-bin-dir/lmplz

# data
#myrk-data = /home/lar/experiment/my-rk/smt1/t9
#mykc-data = /home/lar/experiment/kachin-myanmar/demo-mykc-smt/4demo/
mykc-data = /home/lar/experiment/kachin-myanmar/demo-mykc-smt/4demo2/

### basic tools
#
# moses decoder
decoder = $moses-bin-dir/moses

# conversion of phrase table into binary on-disk format
#ttable-binarizer = $moses-bin-dir/processPhraseTable
#ttable-binarizer = $moses-bin-dir/processPhraseTableMin

# conversion of rule table into binary on-disk format
#ttable-binarizer = "$moses-bin-dir/CreateOnDiskPt 1 1 4 100 2"

# tokenizers - comment out if all your data is already tokenized
#input-tokenizer = "$moses-script-dir/tokenizer/tokenizer.perl -a -l $input-
extension"
#output-tokenizer = "$moses-script-dir/tokenizer/tokenizer.perl -a -l $output-

```

```

extension"

# truecasers - comment out if you do not use the truecaser
#input-truecaser = $moses-script-dir/recaser/truecase.perl
#output-truecaser = $moses-script-dir/recaser/truecase.perl
#dettruecaser = $moses-script-dir/recaser/dettruecase.perl

### generic parallelizer for cluster and multi-core machines
# you may specify a script that allows the parallel execution
# parallizable steps (see meta file). you also need specify
# the number of jobs (cluster) or cores (multicore)
#
generic-parallelizer = $moses-script-dir/ems/support/generic-parallelizer.perl
#generic-parallelizer = $moses-script-dir/ems/support/generic-multicore-
parallelizer.perl

### cluster settings (if run on a cluster machine)
# number of jobs to be submitted in parallel
#
#jobs = 10

# arguments to qsub when scheduling a job
#qsub-settings = ""

# project for privileges and usage accounting
#qsub-project = iccs_smt

# memory and time
#qsub-memory = 4
#qsub-hours = 48

### multi-core settings
# when the generic parallelizer is used, the number of cores
# specified here
#cores = 8

#####
# PARALLEL CORPUS PREPARATION:
# create a tokenized, sentence-aligned corpus, ready for training

[CORPUS]

### long sentences are filtered out, since they slow down GIZA++
# and are a less reliable source of data. set here the maximum
# length of a sentence
#
max-sentence-length = 80

```

[CORPUS:mykc]

```
### command to run to get raw corpus files
#
# get-corpus-script =

### raw corpus files (untokenized, but sentence aligned)
#
raw-stem = $mykc-data/train

### tokenized corpus files (may contain long sentences)
#
#tokenized-stem = $mykc-data/train

### if sentence filtering should be skipped,
# point to the clean training data
#
#clean-stem =
```

```
### if corpus preparation should be skipped,
# point to the prepared training data
#
#lowercased-stem =
```

```
#####
# LANGUAGE MODEL TRAINING
```

[LM]

```
### tool to be used for language model training
# srilm
#lm-training = /data/lttools/training/srilm/srilm-1.6.0/bin/i686-m64/ngram-count
#lm-training = /home/ros/tool/srilm-1.7.1/bin/i686-m64/ngram-count
#settings = "-interpolate -kndiscount -unk"

# firstlm training
# msb = modified kneser ney; p=0 no singleton pruning
#lm-training = "$moses-script-dir/generic/trainlm-first2.perl -cores $cores
-first-dir $firstlm-dir -temp-dir $working-dir/tmp"
#settings = "-s msb -p 0"

# kenlm
# additional parameters to lmplz (check lmplz help message)
#settings = "-T $working-dir/tmp -S 10G"
# this tells EMS to use lmplz and tells EMS where lmplz is located
#lm-training = "$moses-script-dir/generic/trainlm-lmplz.perl -lmplz $lmplz"
#lm-training = "$moses-script-dir/ems/support/lmplz-wrapper.perl -bin $moses-
bin-dir/lmplz"
```

```

# kenlm training
lm-training = "$moses-script-dir/ems/support/lmplz-wrapper.perl -bin $moses-bin-dir/lmplz"
settings = "--prune '0 0 1' -T $working-dir/lm -S 20% --discount_fallback"

#lm-binarizer = $moses-bin-dir/build_binary

# order of the language model
order = 5

### tool to be used for training randomized language model from scratch
# (more commonly, a SRILM is trained)
#
#rlm-training = "$randlm-dir/buildlm -falsepos 8 -values 8"

### script to use for binary table format for irstlm or kenlm
# (default: no binarization)

# irstlm
#lm-binarizer = $irstlm-dir/compile-lm

# kenlm, also set type to 8
lm-binarizer = $moses-bin-dir/build_binary
type = 8

### script to create quantized language model format (irstlm)
# (default: no quantization)
#
#lm-quantizer = $irstlm-dir/quantize-lm

### script to use for converting into randomized table format
# (default: no randomization)
#
#lm-randomizer = "$randlm-dir/buildlm -falsepos 8 -values 8"

### each language model to be used has its own section here

[LM:mykc]

### command to run to get raw corpus files
#
#get-corpus-script = ""

### raw corpus (untokenized)
#
raw-corpus = $mykc-data/train.$output-extension

```

```

### tokenized corpus files (may contain long sentences)
#
#tokenized-corpus = $mykc-data/train.$output-extension

### if corpus preparation should be skipped,
# point to the prepared language model
#
#lm =

#####
# INTERPOLATING LANGUAGE MODELS

[INTERPOLATED-LM]

# if multiple language models are used, these may be combined
# by optimizing perplexity on a tuning set
# see, for instance [Koehn and Schwenk, IJCNLP 2008]

### script to interpolate language models
# if commented out, no interpolation is performed
#
# script = $moses-script-dir/ems/support/interpolate-lm.perl

### tuning set
# you may use the same set that is used for mert tuning (reference set)
#
#tuning-smg =
#raw-tuning =
#tokenized-tuning =
#factored-tuning =
#lowercased-tuning =
#split-tuning =

### group language models for hierarchical interpolation
# (flat interpolation is limited to 10 language models)
#group = "first,second fourth,fifth"

### script to use for binary table format for firstlm or kenlm
# (default: no binarization)

# firstlm
#lm-binarizer = $firstlm-dir/compile-lm

# kenlm, also set type to 8
lm-binarizer = $moses-bin-dir/build_binary
type = 8

```

```

### script to create quantized language model format (irstlm)
# (default: no quantization)
#
#lm-quantizer = $irstlm-dir/quantize-lm

### script to use for converting into randomized table format
# (default: no randomization)
#
#lm-randomizer = "$randlm-dir/buildlm -falsepos 8 -values 8"

#####
# MODIFIED MOORE LEWIS FILTERING

[MML] IGNORE

### specifications for language models to be trained
#
#lm-training = $srilm-dir/ngram-count
#lm-settings = "-interpolate -kndiscount -unk"
#lm-binarizer = $moses-src-dir/bin/build_binary
#lm-query = $moses-src-dir/bin/query
#order = 5

### in-/out-of-domain source/target corpora to train the 4 language model
#
# in-domain: point either to a parallel corpus
#outdomain-stem = [CORPUS:toy:clean-split-stem]

# ... or to two separate monolingual corpora
#indomain-target = [LM:toy:lowercased-corpus]
#raw-indomain-source = $toy-data/nc-5k.$input-extension

# point to out-of-domain parallel corpus
#outdomain-stem = [CORPUS:giga:clean-split-stem]

# settings: number of lines sampled from the corpora to train each language
model on
# (if used at all, should be small as a percentage of corpus)
#settings = "--line-count 100000"

#####
# TRANSLATION MODEL TRAINING

[TRAINING]

### training script to be used: either a legacy script or
# current moses training script (default)
#

```

```

script = $moses-script-dir/training/train-model.perl

### general options
# these are options that are passed on to train-model.perl, for instance
# * "-mgiza -mgiza-cpus 8" to use mgiza instead of giza
# * "-sort-buffer-size 8G -sort-compress gzip" to reduce on-disk sorting
# * "-sort-parallel 8 -cores 8" to speed up phrase table building
#
#training-options = "-sort-parallel 8 -cores 8"

#training-options = "-cores 8"

### factored training: specify here which factors used
# if none specified, single factor training is assumed
# (one translation step, surface to surface)
#
#input-factors = word lemma pos morph
#output-factors = word lemma pos
#alignment-factors = "word -> word"
#translation-factors = "word -> word"
#reordering-factors = "word -> word"
#generation-factors = "word -> pos"
#decoding-steps = "t0, g0"

### parallelization of data preparation step
# the two directions of the data preparation can be run in parallel
# comment out if not needed
#
parallel = yes

### pre-computation for giza++
# giza++ has a more efficient data structure that needs to be
# initialized with snt2cooc. if run in parallel, this may reduces
# memory requirements. set here the number of parts
#
#run-giza-in-parts = 5

### symmetrization method to obtain word alignments from giza output
# (commonly used: grow-diag-final-and)
#
alignment-symmetrization-method = grow-diag-final-and

### use of Chris Dyer's fast align for word alignment
#
#fast-align-settings = "-d -o -v"

### use of berkeley aligner for word alignment
#

```

```

#use-berkeley = true
#alignment-symmetrization-method = berkeley
#berkeley-train = $moses-script-dir/ems/support/berkeley-train.sh
#berkeley-process = $moses-script-dir/ems/support/berkeley-process.sh
#berkeley-jar = /your/path/to/berkeleyaligner-1.1/berkeleyaligner.jar
#berkeley-java-options = "-server -mx30000m -ea"
#berkeley-training-options = "-Main.iters 5 5 -EMWordAligner.numThreads 8"
#berkeley-process-options = "-EMWordAligner.numThreads 8"
#berkeley-posterior = 0.5

### use of baseline alignment model (incremental training)
#
#baseline = 68
#baseline-alignment-model = "$working-dir/training/prepared.$baseline/$input-
extension.vcb \
# $working-dir/training/prepared.$baseline/$output-extension.vcb \
# $working-dir/training/giza.$baseline/${output-extension}-$input-
extension.cooc \
# $working-dir/training/giza-inverse.$baseline/${input-extension}-$output-
extension.cooc \
# $working-dir/training/giza.$baseline/${output-extension}-$input-
extension.thmm.5 \
# $working-dir/training/giza.$baseline/${output-extension}-$input-
extension.hhmm.5 \
# $working-dir/training/giza-inverse.$baseline/${input-extension}-$output-
extension.thmm.5 \
# $working-dir/training/giza-inverse.$baseline/${input-extension}-$output-
extension.hhmm.5"

### if word alignment should be skipped,
# point to word alignment files
#
#word-alignment = $working-dir/model/aligned.1

### filtering some corpora with modified Moore-Lewis
# specify corpora to be filtered and ratio to be kept, either before or after
word alignment
#mml-filter-corpora = toy
#mml-before-wa = "-proportion 0.9"
#mml-after-wa = "-proportion 0.9"

### create a bilingual concordancer for the model
#
#biconcor = $moses-script-dir/ems/biconcor/biconcor

#####

### use of Operation Sequence Model

```

```

### Durrani, Schmid and Fraser. (2011): "A Joint Sequence Translation Model with
Integrated Reordering"

#operation-sequence-model = "yes"
#operation-sequence-model-order = 5
#operation-sequence-model-settings = ""

### compile Moses with --max-kenlm-order=9 if higher order is required

### if OSM training should be skipped,
# point to OSM Model
#
# osm-model =

#####

### lexicalized reordering: specify orientation type
# (default: only distance-based reordering model)
#
lexicalized-reordering = msd-bidirectional-fe

### hierarchical rule set
#
#hierarchical-rule-set = true

### settings for rule extraction
#
#extract-settings = ""
max-phrase-length = 5

### add extracted phrases from baseline model
#
#baseline-extract = $working-dir/model/extract.$baseline
#
# requires aligned parallel corpus for re-estimating lexical translation
probabilities
#baseline-corpus = $working-dir/training/corpus.$baseline
#baseline-alignment = $working-dir/model/aligned.$baseline.$alignment-
symmetrization-method

### unknown word labels (target syntax only)
# enables use of unknown word labels during decoding
# label file is generated during rule extraction
#
#use-unknown-word-labels = true

### if phrase extraction should be skipped,

```

```

# point to stem for extract files
#
# extracted-phrases =

### settings for rule scoring
#
score-settings = "--GoodTuring"

### include word alignment in phrase table
#
#include-word-alignment-in-rules = yes

### sparse lexical features
#
#sparse-features = "target-word-insertion top 50, source-word-deletion top 50,
word-translation top 50 50, phrase-length"

### domain adaptation settings
# options: sparse, any of: indicator, subset, ratio
#domain-features = "subset"

### if phrase table training should be skipped,
# point to phrase translation table
#
# phrase-translation-table =

### if reordering table training should be skipped,
# point to reordering table
#
# reordering-table =

### filtering the phrase table based on significance tests
# Johnson, Martin, Foster and Kuhn. (2007): "Improving Translation Quality by
Discarding Most of the Phrasetable"
# options: -n number of translations; -l 'a+e', 'a-e', or a positive real value
-log prob threshold
#salm-index = /path/to/project/salm/Bin/Linux/Index/IndexSA.064
#sigtest-filter = "-l a+e -n 50"

### if training should be skipped,
# point to a configuration file that contains
# pointers to all relevant model files
#
#config-with-reused-weights =

#####
### TUNING: finding good weights for model components

```

[TUNING]

```
### instead of tuning with this setting, old weights may be recycled
# specify here an old configuration file with matching weights
#
#weight-config = /panfs/panltg2/users/finch/expt/moses2.1/plain.weight.ini

### tuning script to be used
#
tuning-script = $moses-script-dir/training/mert-moses.pl
tuning-settings = "-mertdir $moses-bin-dir"

### specify the corpus used for tuning
# it should contain 1000s of sentences
#
#input-sm =
raw-input = $mykc-data/dev.$input-extension

#tokenized-input =
#factorized-input =
#input =
#
#reference-sm =
raw-reference = $mykc-data/dev.$output-extension

#tokenized-reference =
#factorized-reference =
#reference =

### size of n-best list used (typically 100)
#
nbest = 100

### ranges for weights for random initialization
# if not specified, the tuning script will use generic ranges
# it is not clear, if this matters
#
# lambda =

### additional flags for the filter script
#
filter-settings = ""

### additional flags for the decoder
#
decoder-settings = ""

### if tuning should be skipped, specify this here
```

```

# and also point to a configuration file that contains
# pointers to all relevant model files
#
#config =

#####
## RECASER: restore case, this part only trains the model

[RECASING]

#decoder = $moses-bin-dir/moses

### training data
# raw input needs to be still tokenized,
# also also tokenized input may be specified
#
#tokenized = [LM:europarl:tokenized-corpus]

# recase-config =

#lm-training = $srilm-dir/ngram-count

#####
## TRUECASER: train model to truecase corpora and input

[TRUECASER]

### script to train truecaser models
#
#trainer = $moses-script-dir/recaser/train-truecaser.perl

### training data
# data on which truecaser is trained
# if no training data is specified, parallel corpus is used
#
# raw-stem =
# tokenized-stem =

### trained model
#
# truecase-model =

#####
## EVALUATION: translating a test set using the tuned system and score it

[EVALUATION]

### additional flags for the filter script

```

```

#
#filter-settings = ""

### additional decoder settings
# switches for the Moses decoder
# common choices:
#   "-threads N" for multi-threading
#   "-mbr" for MBR decoding
#   "-drop-unknown" for dropping unknown source words
#   "-search-algorithm 1 -cube-pruning-pop-limit 5000 -s 5000" for cube pruning
#
decoder-settings = "-search-algorithm 1 -cube-pruning-pop-limit 5000 -s 5000"

### specify size of n-best list, if produced
#
#nbest = 100

### multiple reference translations
#
#multiref = yes

### prepare system output for scoring
# this may include detokenization and wrapping output in sgm
# (needed for nist-bleu, ter, meteor)
#
#detokenizer = "$moses-script-dir/tokenizer/detokenizer.perl -l $output-
extension"
#recaser = $moses-script-dir/recaser/recase.perl
wrapping-script = "$moses-script-dir/ems/support/wrap-xml.perl $output-
extension"
#output-sgm =

### BLEU
#
nist-bleu = $moses-script-dir/generic/mteval-v13a.pl
nist-bleu-c = "$moses-script-dir/generic/mteval-v13a.pl -c"
#multi-bleu = $moses-script-dir/generic/multi-bleu.perl
#ibm-bleu =

### TER: translation error rate (BBN metric) based on edit distance
# not yet integrated
#
# ter =

### METEOR: gives credit to stem / worknet synonym matches
# not yet integrated
#
# meteor =

```

```

### Analysis: carry out various forms of analysis on the output
#
analysis = $moses-script-dir/ems/support/analysis.perl
#
# also report on input coverage
analyze-coverage = yes
#
# also report on phrase mappings used
report-segmentation = yes
#
# report precision of translations for each input word, broken down by
# count of input word in corpus and model
report-precision-by-coverage = yes
#
# further precision breakdown by factor
#precision-by-coverage-factor = pos
#
# visualization of the search graph in tree-based models
#analyze-search-graph = yes

[EVALUATION:test]

### input data
#
#input-smg = /panfs/panmt/users/ye/cluster/test-smg/test.$input-
extension.src.sgm
#input-smg = $myrk-data/test-smg/test.$input-extension.src.sgm
input-smg = $mykc-data/test-smg/test.$input-extension.src.sgm

# raw-input = $mykc-data/test.$input-extension
# tokenized-input =
# factorized-input =
# input =

### reference data
#
#reference-smg = /panfs/panmt/users/ye/cluster/test-smg/test.$output-
extension.ref.sgm
#reference-smg = $myrk-data/test-smg/test.$output-extension.ref.sgm
reference-smg = $mykc-data/test-smg/test.$output-extension.ref.sgm

# raw-reference = $mykc-data/test.$output-extension
# tokenized-reference =
# reference =

### analysis settings
# may contain any of the general evaluation analysis settings

```

```
# specific setting: base coverage statistics on earlier run
#
#precision-by-coverage-base = $working-dir/evaluation/test.analysis.5

### wrapping frame
# for nist-bleu and other scoring scripts, the output needs to be wrapped
# in sgm markup (typically like the input sgm)
#
wrapping-frame = $input-sgm

#####
### REPORTING: summarize evaluation scores
```

[REPORTING]

```
### currently no parameters for reporting section
```

Original config.baseline ကို ဝင်ပြင်ပါ။ ပြီးရင် အောက်ပါအတိုင်း run ပြီး Thai-Myamar အတွက် config
ဖိုင်ကို generate လုပ်ပါ။

[161]:

```
!ls

config.baseline  generate_config.pl  run-baseline.pl  run-pbsmt.sh

ပြင်ပြီးသွားတဲ့ config.baseline ဖိုင်က အောက်ပါအတိုင်းပါ
```

[162]:

```
!cat config.baseline

### directories that contain tools and data
#
# moses
#moses-src-dir = /home/ros/mosesdecoder
#moses-src-dir = /home/lar/tool/moses/
moses-src-dir = /home/ye/tool/mosesbin/ubuntu-17.04/moses

# moses binaries
moses-bin-dir = $moses-src-dir/bin

# moses scripts
moses-script-dir = $moses-src-dir/scripts

# directory where GIZA++/MGIZA programs resides
#external-bin-dir = /home/lar/tool/giza-pp-master/mkcls-v2
#external-bin-dir = /home/lar/tool/giza-pp-master/GIZA++-v2
#external-bin-dir = /home/lar/tool/giza-pp-master/
#external-bin-dir = /home/ye/tool/mgiza/mgizapp/bin
external-bin-dir = /home/ye/tool/giza-pp/GIZA++-v2
```

```

# srilm
#srilm-dir = /data/lttools/training/srilm/srilm-1.6.0/bin/i686-m64
#I haven't installed SRILM yet on Deep Learning Box computer
#srilm-dir = /home/ros/tool/srilm-1.7.1/bin/i686-m64

# irstlm
#irstlm-dir = $moses-src-dir/irstlm/bin

# randlm
#randlm-dir = $moses-src-dir/randlm/bin

# kenlm
lmplz = $moses-bin-dir/lmplz

# data
#myrk-data = /home/lar/experiment/my-rk/smt1/t9
#mykc-data = /home/lar/experiment/kachin-myanmar/demo-mykc-smt/4demo/
#mykc-data = /home/lar/experiment/kachin-myanmar/demo-mykc-smt/4demo2/
#myrk-data = /media/lar/Transcend/student/lecture/mtrss/pbsmt-demo/pbsmt/data/
thmy-data = /home/ye/exp/SMT-NMT_tutorial/clean-data/

### basic tools
#
# moses decoder
decoder = $moses-bin-dir/moses

# conversion of phrase table into binary on-disk format
#ttable-binarizer = $moses-bin-dir/processPhraseTable
#ttable-binarizer = $moses-bin-dir/processPhraseTableMin

# conversion of rule table into binary on-disk format
#ttable-binarizer = "$moses-bin-dir/CreateOnDiskPt 1 1 4 100 2"

# tokenizers - comment out if all your data is already tokenized
#input-tokenizer = "$moses-script-dir/tokenizer/tokenizer.perl -a -l $input-
extension"
#output-tokenizer = "$moses-script-dir/tokenizer/tokenizer.perl -a -l $output-
extension"

# truecasers - comment out if you do not use the truecaser
#input-truecaser = $moses-script-dir/recaser/truecase.perl
#output-truecaser = $moses-script-dir/recaser/truecase.perl
#dettruecaser = $moses-script-dir/recaser/dettruecase.perl

### generic parallelizer for cluster and multi-core machines
# you may specify a script that allows the parallel execution
# parallizable steps (see meta file). you also need specify

```

```

# the number of jobs (cluster) or cores (multicore)
#
generic-parallelizer = $moses-script-dir/ems/support/generic-parallelizer.perl
#generic-parallelizer = $moses-script-dir/ems/support/generic-multicore-
parallelizer.perl

### cluster settings (if run on a cluster machine)
# number of jobs to be submitted in parallel
#
#jobs = 10

# arguments to qsub when scheduling a job
#qsub-settings = ""

# project for privileges and usage accounting
#qsub-project = iccs_smt

# memory and time
#qsub-memory = 4
#qsub-hours = 48

### multi-core settings
# when the generic parallelizer is used, the number of cores
# specified here
#cores = 8

#####
# PARALLEL CORPUS PREPARATION:
# create a tokenized, sentence-aligned corpus, ready for training

[CORPUS]

### long sentences are filtered out, since they slow down GIZA++
# and are a less reliable source of data. set here the maximum
# length of a sentence
#
max-sentence-length = 80

[CORPUS:thmy]

### command to run to get raw corpus files
#
# get-corpus-script =

### raw corpus files (untokenized, but sentence aligned)
#
raw-stem = $myrk-data/train

```

```

### tokenized corpus files (may contain long sentences)
#
#tokenized-stem = $myrk-data/train

### if sentence filtering should be skipped,
# point to the clean training data
#
#clean-stem =

### if corpus preparation should be skipped,
# point to the prepared training data
#
#lowercased-stem =

#####
# LANGUAGE MODEL TRAINING

[LM]

### tool to be used for language model training
# srilm
#lm-training = /data/lttools/training/srilm/srilm-1.6.0/bin/i686-m64/ngram-count
#lm-training = /home/ros/tool/srilm-1.7.1/bin/i686-m64/ngram-count
#settings = "-interpolate -kndiscount -unk"

# irstlm training
# msb = modified kneser ney; p=0 no singleton pruning
#lm-training = "$moses-script-dir/generic/trainlm-irst2.perl -cores $cores
-first-dir $irstlm-dir -temp-dir $working-dir/tmp"
#settings = "-s msb -p 0"

# kenlm
# additional parameters to lmplz (check lmplz help message)
#settings = "-T $working-dir/tmp -S 10G"
# this tells EMS to use lmplz and tells EMS where lmplz is located
#lm-training = "$moses-script-dir/generic/trainlm-lmplz.perl -lmplz $lmplz"
#lm-training = "$moses-script-dir/ems/support/lmplz-wrapper.perl -bin $moses-
bin-dir/lmplz"

# kenlm training
lm-training = "$moses-script-dir/ems/support/lmplz-wrapper.perl -bin $moses-bin-
dir/lmplz"
settings = "--prune '0 0 1' -T $working-dir/lm -S 20% --discount_fallback"

#lm-binarizer = $moses-bin-dir/build_binary

# order of the language model

```

```

order = 5

### tool to be used for training randomized language model from scratch
# (more commonly, a SRILM is trained)
#
#rlm-training = "$randlm-dir/buildlm -falsepos 8 -values 8"

### script to use for binary table format for irstlm or kenlm
# (default: no binarization)

# irstlm
#lm-binarizer = $irstlm-dir/compile-lm

# kenlm, also set type to 8
lm-binarizer = $moses-bin-dir/build_binary
type = 8

### script to create quantized language model format (irstlm)
# (default: no quantization)
#
#lm-quantizer = $irstlm-dir/quantize-lm

### script to use for converting into randomized table format
# (default: no randomization)
#
#lm-randomizer = "$randlm-dir/buildlm -falsepos 8 -values 8"

### each language model to be used has its own section here

[LM:thmy]

### command to run to get raw corpus files
#
#get-corpus-script = ""

### raw corpus (untokenized)
#
raw-corpus = $myrk-data/train.$output-extension

### tokenized corpus files (may contain long sentences)
#
#tokenized-corpus = $my-data/train.$output-extension

### if corpus preparation should be skipped,
# point to the prepared language model
#
#lm =

```

```
#####
# INTERPOLATING LANGUAGE MODELS

[INTERPOLATED-LM]

# if multiple language models are used, these may be combined
# by optimizing perplexity on a tuning set
# see, for instance [Koehn and Schwenk, IJCNLP 2008]

### script to interpolate language models
# if commented out, no interpolation is performed
#
# script = $moses-script-dir/ems/support/interpolate-lm.perl

### tuning set
# you may use the same set that is used for mert tuning (reference set)
#
#tuning-sgm =
#raw-tuning =
#tokenized-tuning =
#factored-tuning =
#lowercased-tuning =
#split-tuning =

### group language models for hierarchical interpolation
# (flat interpolation is limited to 10 language models)
#group = "first,second fourth,fifth"

### script to use for binary table format for irstlm or kenlm
# (default: no binarization)

# irstlm
#lm-binarizer = $irstlm-dir/compile-lm

# kenlm, also set type to 8
lm-binarizer = $moses-bin-dir/build_binary
type = 8

### script to create quantized language model format (irstlm)
# (default: no quantization)
#
#lm-quantizer = $irstlm-dir/quantize-lm

### script to use for converting into randomized table format
# (default: no randomization)
#
#lm-randomizer = "$randlm-dir/buildlm -falsepos 8 -values 8"
```

```
#####
# MODIFIED MOORE LEWIS FILTERING

[MML] IGNORE

### specifications for language models to be trained
#
#lm-training = $srilm-dir/ngram-count
#lm-settings = "-interpolate -kndiscount -unk"
#lm-binarizer = $moses-src-dir/bin/build_binary
#lm-query = $moses-src-dir/bin/query
#order = 5

### in-/out-of-domain source/target corpora to train the 4 language model
#
# in-domain: point either to a parallel corpus
#outdomain-stem = [CORPUS:toy:clean-split-stem]

# ... or to two separate monolingual corpora
#indomain-target = [LM:toy:lowercased-corpus]
#raw-indomain-source = $toy-data/nc-5k.$input-extension

# point to out-of-domain parallel corpus
#outdomain-stem = [CORPUS:giga:clean-split-stem]

# settings: number of lines sampled from the corpora to train each language
model on
# (if used at all, should be small as a percentage of corpus)
#settings = "--line-count 100000"

#####
# TRANSLATION MODEL TRAINING

[TRAINING]

### training script to be used: either a legacy script or
# current mooses training script (default)
#
script = $moses-script-dir/training/train-model.perl

### general options
# these are options that are passed on to train-model.perl, for instance
# * "-mgiza -mgiza-cpus 8" to use mgiza instead of giza
# * "-sort-buffer-size 8G -sort-compress gzip" to reduce on-disk sorting
# * "-sort-parallel 8 -cores 8" to speed up phrase table building
#
#training-options = "-sort-parallel 8 -cores 8"
```

```

#training-options = "-cores 8"

### factored training: specify here which factors used
# if none specified, single factor training is assumed
# (one translation step, surface to surface)
#
#input-factors = word lemma pos morph
#output-factors = word lemma pos
#alignment-factors = "word -> word"
#translation-factors = "word -> word"
#reordering-factors = "word -> word"
#generation-factors = "word -> pos"
#decoding-steps = "t0, g0"

### parallelization of data preparation step
# the two directions of the data preparation can be run in parallel
# comment out if not needed
#
parallel = yes

### pre-computation for giza++
# giza++ has a more efficient data structure that needs to be
# initialized with snt2cooc. if run in parallel, this may reduces
# memory requirements. set here the number of parts
#
#run-giza-in-parts = 5

### symmetrization method to obtain word alignments from giza output
# (commonly used: grow-diag-final-and)
#
alignment-symmetrization-method = grow-diag-final-and

### use of Chris Dyer's fast align for word alignment
#
#fast-align-settings = "-d -o -v"

### use of berkeley aligner for word alignment
#
#use-berkeley = true
#alignment-symmetrization-method = berkeley
#berkeley-train = $moses-script-dir/ems/support/berkeley-train.sh
#berkeley-process = $moses-script-dir/ems/support/berkeley-process.sh
#berkeley-jar = /your/path/to/berkeleyaligner-1.1/berkeleyaligner.jar
#berkeley-java-options = "-server -mx30000m -ea"
#berkeley-training-options = "-Main.iters 5 5 -EMWordAligner.numThreads 8"
#berkeley-process-options = "-EMWordAligner.numThreads 8"
#berkeley-posterior = 0.5

```

```

### use of baseline alignment model (incremental training)
#
#baseline = 68
#baseline-alignment-model = "$working-dir/training/prepared.$baseline/$input-
extension.vcb \
# $working-dir/training/prepared.$baseline/$output-extension.vcb \
# $working-dir/training/giza.$baseline/${output-extension}-$input-
extension.cooc \
# $working-dir/training/giza-inverse.$baseline/${input-extension}-$output-
extension.cooc \
# $working-dir/training/giza.$baseline/${output-extension}-$input-
extension.thmm.5 \
# $working-dir/training/giza.$baseline/${output-extension}-$input-
extension.hhmm.5 \
# $working-dir/training/giza-inverse.$baseline/${input-extension}-$output-
extension.thmm.5 \
# $working-dir/training/giza-inverse.$baseline/${input-extension}-$output-
extension.hhmm.5"

### if word alignment should be skipped,
# point to word alignment files
#
#word-alignment = $working-dir/model/aligned.1

### filtering some corpora with modified Moore-Lewis
# specify corpora to be filtered and ratio to be kept, either before or after
word alignment
#mml-filter-corpora = toy
#mml-before-wa = "-proportion 0.9"
#mml-after-wa = "-proportion 0.9"

### create a bilingual concordancer for the model
#
#biconcor = $moses-script-dir/ems/biconcor/biconcor

#####

### use of Operation Sequence Model
### Durrani, Schmid and Fraser. (2011): "A Joint Sequence Translation Model with
Integrated Reordering"

#operation-sequence-model = "yes"
#operation-sequence-model-order = 5
#operation-sequence-model-settings = ""

### compile Moses with --max-kenlm-order=9 if higher order is required

### if OSM training should be skipped,

```

```

# point to OSM Model
#
# osm-model =

#####

### lexicalized reordering: specify orientation type
# (default: only distance-based reordering model)
#
lexicalized-reordering = msd-bidirectional-fe

### hierarchical rule set
#
#hierarchical-rule-set = true

### settings for rule extraction
#
#extract-settings = ""
max-phrase-length = 5

### add extracted phrases from baseline model
#
#baseline-extract = $working-dir/model/extract.$baseline
#
# requires aligned parallel corpus for re-estimating lexical translation
probabilities
#baseline-corpus = $working-dir/training/corpus.$baseline
#baseline-alignment = $working-dir/model/aligned.$baseline.$alignment-
symmetrization-method

### unknown word labels (target syntax only)
# enables use of unknown word labels during decoding
# label file is generated during rule extraction
#
#use-unknown-word-labels = true

### if phrase extraction should be skipped,
# point to stem for extract files
#
# extracted-phrases =

### settings for rule scoring
#
score-settings = "--GoodTuring"

### include word alignment in phrase table
#

```

```

#include-word-alignment-in-rules = yes

### sparse lexical features
#
#sparse-features = "target-word-insertion top 50, source-word-deletion top 50,
word-translation top 50 50, phrase-length"

### domain adaptation settings
# options: sparse, any of: indicator, subset, ratio
#domain-features = "subset"

### if phrase table training should be skipped,
# point to phrase translation table
#
# phrase-translation-table =

### if reordering table training should be skipped,
# point to reordering table
#
# reordering-table =

### filtering the phrase table based on significance tests
# Johnson, Martin, Foster and Kuhn. (2007): "Improving Translation Quality by
Discarding Most of the Phrasetable"
# options: -n number of translations; -l 'a+e', 'a-e', or a positive real value
-log prob threshold
#salm-index = /path/to/project/salm/Bin/Linux/Index/IndexSA.064
#sigtest-filter = "-l a+e -n 50"

### if training should be skipped,
# point to a configuration file that contains
# pointers to all relevant model files
#
#config-with-reused-weights =

#####
### TUNING: finding good weights for model components

[TUNING]

### instead of tuning with this setting, old weights may be recycled
# specify here an old configuration file with matching weights
#
#weight-config = /panfs/panltg2/users/finch/expt/moses2.1/plain.weight.ini

### tuning script to be used
#
tuning-script = $moses-script-dir/training/mert-moses.pl

```

```

tuning-settings = "-mertdir $moses-bin-dir"

### specify the corpus used for tuning
# it should contain 1000s of sentences
#
#input-smg =
raw-input = $myrk-data/dev.$input-extension

#tokenized-input =
#factorized-input =
#input =
#
#reference-smg =
raw-reference = $myrk-data/dev.$output-extension

#tokenized-reference =
#factorized-reference =
#reference =

### size of n-best list used (typically 100)
#
nbest = 100

### ranges for weights for random initialization
# if not specified, the tuning script will use generic ranges
# it is not clear, if this matters
#
# lambda =

### additional flags for the filter script
#
filter-settings = ""

### additional flags for the decoder
#
decoder-settings = ""

### if tuning should be skipped, specify this here
# and also point to a configuration file that contains
# pointers to all relevant model files
#
#config =

#####
## RECASER: restore case, this part only trains the model

[RECASING]

```

```

#decoder = $moses-bin-dir/moses

### training data
# raw input needs to be still tokenized,
# also also tokenized input may be specified
#
#tokenized = [LM:europarl:tokenized-corpus]

# recase-config =

#lm-training = $srilm-dir/ngram-count

#####
## TRUECASER: train model to truecase corpora and input

[TRUECASER]

### script to train truecaser models
#
#trainer = $moses-script-dir/recaser/train-truecaser.perl

### training data
# data on which truecaser is trained
# if no training data is specified, parallel corpus is used
#
# raw-stem =
# tokenized-stem =

### trained model
#
# truecase-model =

#####
## EVALUATION: translating a test set using the tuned system and score it

[EVALUATION]

### additional flags for the filter script
#
#filter-settings = ""

### additional decoder settings
# switches for the Moses decoder
# common choices:
#   "-threads N" for multi-threading
#   "-mbr" for MBR decoding
#   "-drop-unknown" for dropping unknown source words
#   "-search-algorithm 1 -cube-pruning-pop-limit 5000 -s 5000" for cube pruning

```

```

#
decoder-settings = "-search-algorithm 1 -cube-pruning-pop-limit 5000 -s 5000"

### specify size of n-best list, if produced
#
#nbest = 100

### multiple reference translations
#
#multiref = yes

### prepare system output for scoring
# this may include detokenization and wrapping output in sgm
# (needed for nist-bleu, ter, meteor)
#
#detokenizer = "$moses-script-dir/tokenizer/detokenizer.perl -l $output-
extension"
#recaser = $moses-script-dir/recaser/recase.perl
wrapping-script = "$moses-script-dir/ems/support/wrap-xml.perl $output-
extension"
#output-sgm =

### BLEU
#
nist-bleu = $moses-script-dir/generic/mteval-v13a.pl
nist-bleu-c = "$moses-script-dir/generic/mteval-v13a.pl -c"
#multi-bleu = $moses-script-dir/generic/multi-bleu.perl
#ibm-bleu =

### TER: translation error rate (BBN metric) based on edit distance
# not yet integrated
#
# ter =

### METEOR: gives credit to stem / worknet synonym matches
# not yet integrated
#
# meteor =

### Analysis: carry out various forms of analysis on the output
#
analysis = $moses-script-dir/ems/support/analysis.perl
#
# also report on input coverage
analyze-coverage = yes
#
# also report on phrase mappings used
report-segmentation = yes

```

```

#
# report precision of translations for each input word, broken down by
# count of input word in corpus and model
report-precision-by-coverage = yes
#
# further precision breakdown by factor
#precision-by-coverage-factor = pos
#
# visualization of the search graph in tree-based models
#analyze-search-graph = yes

[EVALUATION:test]

### input data
#
#input-smg = /panfs/panmt/users/ye/cluster/test-smg/test.$input-
extension.src.sgm
input-smg = $myrk-data/test-smg/test.$input-extension.src.sgm
#input-smg = $mykc-data/test-smg/test.$input-extension.src.sgm

# raw-input = $mykc-data/test.$input-extension
# tokenized-input =
# factorized-input =
# input =

### reference data
#
#reference-smg = /panfs/panmt/users/ye/cluster/test-smg/test.$output-
extension.ref.sgm
reference-smg = $myrk-data/test-smg/test.$output-extension.ref.sgm
#reference-smg = $mykc-data/test-smg/test.$output-extension.ref.sgm

# raw-reference = $mykc-data/test.$output-extension
# tokenized-reference =
# reference =

### analysis settings
# may contain any of the general evaluation analysis settings
# specific setting: base coverage statistics on earlier run
#
#precision-by-coverage-base = $working-dir/evaluation/test.analysis.5

### wrapping frame
# for nist-bleu and other scoring scripts, the output needs to be wrapped
# in smg markup (typically like the input smg)
#
wrapping-frame = $input-smg

```

```
#####  
### REPORTING: summarize evaluation scores
```

```
[REPORTING]
```

```
### currently no parameters for reporting section
```

1.10 Updating generate_config.pl

generate_config.pl ဖိုင်ကို ဝင်ပြင်ရအုံးမယ်။

```
[112]: !cat generate_config.pl
```

```
#!/usr/bin/perl
```

```
# Preparing configuration files for PBSMT experiments  
# Output: one configuration file for baseline or PBSMT  
# *** Before you run this script, you should prepare training, development and  
test parallel-data in advance!!!
```

```
# Written by Finch-san and Ye  
# We used this script for running our NAACL paper:  
# Ye Kyaw Thu, Andrew Finch and Eiichiro Sumita,  
# "Interlocking Phrases in Phrase-based Statistical Machine Translation",  
# In Proceedings of the NAACL-HLT 2016, June 12-17, San Diego, US, pp.  
1076-1081.
```

```
# Last updated 9 Oct 2019, @Machine Translation Research Summer School, YTU,  
Myanmar
```

```
use strict;
```

```
my @langs;
```

```
# you have to update the PBSMT experiment path!!!  
#my $smtpath = "/home/lar/experiment/kachin-myanmar/demo-mykc-smt";  
my $smtpath = "/media/lar/Transcend/student/lecture/mtrss/pbsmt-demo/pbsmt";
```

```
# you have to update the data path for running PBSMT experiment!!!
```

```
#foreach my $trainFile (  
</panfs/panltg2/users/finch/expt/multibtec/corpus/train.[a-z][a-z]> )  
#foreach my $trainFile (  
</panfs/panmt/users/ye/cluster/corpus/train.[a-z][a-z]>)  
#foreach my $trainFile ( </home/tha/experiment/my-rk/data/train.[a-z][a-z]>)  
#foreach my $trainFile ( </home/lar/experiment/kachin-myanmar/demo-mykc-  
smt/4demo/train.[a-z][a-z]>)  
foreach my $trainFile ( </media/lar/Transcend/student/lecture/mtrss/pbsmt-
```

```

demo/pbsmt/data/train.[a-z][a-z]>)
{
    $trainFile =~ m/train.([a-z][a-z])/;

    push @langs, $1;
}

foreach my $expt (qw/baseline/)
{
    die("$smtpath/$expt") if (-d "$smtpath/$expt");
    `mkdir $smtpath/$expt`;
    foreach my $src (@langs)
    {
        foreach my $trg (@langs)
        {
            next if $src eq $trg;
            #next if $src ne "my" and $trg ne "my";

            my $pair = "${src}-${trg}";
            my $configFile = "$smtpath/$expt/$pair/config.${expt}.$pair";

            `mkdir $smtpath/$expt/$pair`;

            open FILE, ">$configFile" or die;

            print FILE "# Config file for $pair ($expt)\n";
            print FILE "\n[GENERAL]\n";
            print FILE "working-dir = $smtpath/$expt/$pair\n";
            print FILE "input-extension = ${src}\n";
            print FILE "output-extension = ${trg}\n";
            print FILE "pair-extension = ${pair}\n";
            print FILE "\n";
            close FILE;

            `cat $configFile config.$expt > tmp`;
            `mv tmp $configFile`;
        }
    }
}

```

After updating

Update လုပ်ပြီးသားဖိုင်က အောက်ပါအတိုင်းပါ။

```
[114]: !cat generate_config.pl
```

```
#!/usr/bin/perl
```

```

# Preparing configuration files for PBSMT experiments
# Output: one configuration file for baseline or PBSMT
# *** Before you run this script, you should prepare training, development and
test parallel-data in advance!!!

# Written by Finch-san and Ye
# We used this script for running our NAACL paper:
# Ye Kyaw Thu, Andrew Finch and Eiichiro Sumita,
# "Interlocking Phrases in Phrase-based Statistical Machine Translation",
# In Proceedings of the NAACL-HLT 2016, June 12-17, San Diego, US, pp.
1076-1081.

# Last updated 9 Oct 2019, @Machine Translation Research Summer School, YTU,
Myanmar

use strict;

my @langs;

# you have to update the PBSMT experiment path!!!
my $smtpath = "/home/lar/experiment/kachin-myanmar/demo-mykc-smt";
my $smtpath = "/media/lar/Transcend/student/lecture/mtrss/pbsmt-demo/pbsmt";
my $smtpath = "/home/ye/exp/SMT-NMT_tutorial/pbsmt";

# you have to update the data path for running PBSMT experiment!!!

foreach my $trainFile (
</panfs/panltg2/users/finch/expt/multibtec/corpus/train.[a-z][a-z]> )
foreach my $trainFile (
</panfs/panmt/users/ye/cluster/corpus/train.[a-z][a-z]>)
foreach my $trainFile ( </home/tha/experiment/my-rk/data/train.[a-z][a-z]>)
foreach my $trainFile ( </home/lar/experiment/kachin-myanmar/demo-mykc-
smt/4demo/train.[a-z][a-z]>)
foreach my $trainFile ( </media/lar/Transcend/student/lecture/mtrss/pbsmt-
demo/pbsmt/data/train.[a-z][a-z]>)
foreach my $trainFile ( </home/ye/exp/SMT-NMT_tutorial/clean-
data/train.[a-z][a-z]>)
{
    $trainFile =~ m/train.([a-z][a-z])/;

    push @langs, $1;
}

foreach my $expt (qw/baseline/)
{

die("$smtpath/$expt") if (-d "$smtpath/$expt");
`mkdir $smtpath/$expt`;

```

```

foreach my $src (@langs)
{
    foreach my $trg (@langs)
    {
        next if $src eq $trg;
        #next if $src ne "my" and $trg ne "my";

        my $pair = "${src}-${trg}";
        my $configFile = "$smtpath/$expt/$pair/config.${expt}.$pair";

        `mkdir $smtpath/$expt/$pair`;

        open FILE, ">$configFile" or die;

        print FILE "# Config file for $pair ($expt)\n";
        print FILE "\n[GENERAL]\n";
        print FILE "working-dir = $smtpath/$expt/$pair\n";
        print FILE "input-extension = ${src}\n";
        print FILE "output-extension = ${trg}\n";
        print FILE "pair-extension = ${pair}\n";
        print FILE "\n";
        close FILE;

        `cat $configFile config.$expt > tmp`;
        `mv tmp $configFile`;
    }
}

```

အဓိက ဝင်ပြင်ခဲ့တာက path တွေပါပဲ အထက်ပါအတိုင်း ပြင်ဆင်ပြီးသွားရင် configuration ဖိုင်ကို generate လုပ်ပေးမယ့် perl script ကို run ကြရအောင်။

[115]:

```

!ls

config.baseline      nohup.out            th-my.config.baseline
generate_config.pl    run-baseline.pl
kc-my.config.baseline run-pbsmt.sh

```

[163]:

```

!perl ./generate_config.pl

```

[164]:

```

!ls

baseline config.baseline generate_config.pl run-baseline.pl run-pbsmt.sh

```

[165]:

```

!ls -ad */

baseline/

```

[170]:

```

%cd ..

```

```
/home/ye/exp/SMT-NMT_tutorial/pbsmt
```

```
[171]: !tree .
```

```
.
├── baseline
│   ├── my-th
│   │   └── config.baseline.my-th
│   ├── th-my
│   │   └── config.baseline.th-my
│   ├── config.baseline
│   ├── generate_config.pl
│   ├── run-baseline.pl
│   └── run-pbsmt.sh
```

```
4 directories, 6 files
```

```
[172]: %cd baseline/th-my/
```

```
/home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/th-my
```

```
[173]: cat ./config.baseline.th-my
```

```
# Config file for th-my (baseline)
```

```
[GENERAL]
```

```
working-dir = /home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/th-my
```

```
input-extension = th
```

```
output-extension = my
```

```
pair-extension = th-my
```

```
### directories that contain tools and data
```

```
#
```

```
# moses
```

```
#moses-src-dir = /home/ros/mosesdecoder
```

```
#moses-src-dir = /home/lar/tool/moses/
```

```
moses-src-dir = /home/ye/tool/mosesbin/ubuntu-17.04/moses
```

```
# moses binaries
```

```
moses-bin-dir = $moses-src-dir/bin
```

```
# moses scripts
```

```
moses-script-dir = $moses-src-dir/scripts
```

```
# directory where GIZA++/MGIZA programs resides
```

```
#external-bin-dir = /home/lar/tool/giza-pp-master/mkcls-v2
```

```
#external-bin-dir = /home/lar/tool/giza-pp-master/GIZA++-v2
```

```
#external-bin-dir = /home/lar/tool/giza-pp-master/
```

```

#external-bin-dir = /home/ye/tool/mgiza/mgizapp/bin
external-bin-dir = /home/ye/tool/giza-pp/GIZA++-v2

# srilm
#srilm-dir = /data/lttools/training/srilm/srilm-1.6.0/bin/i686-m64
#I haven't installed SRILM yet on Deep Learning Box computer
#srilm-dir = /home/ros/tool/srilm-1.7.1/bin/i686-m64

#IRSTLM
#irstlm-dir = $moses-src-dir/irstlm/bin

# randlm
#randlm-dir = $moses-src-dir/randlm/bin

# kenlm
lmplz = $moses-bin-dir/lmplz

# data
#myrk-data = /home/lar/experiment/my-rk/smt1/t9
#mykc-data = /home/lar/experiment/kachin-myanmar/demo-mykc-smt/4demo/
#mykc-data = /home/lar/experiment/kachin-myanmar/demo-mykc-smt/4demo2/
#myrk-data = /media/lar/Transcend/student/lecture/mtrss/pbsmt-demo/pbsmt/data/
thmy-data = /home/ye/exp/SMT-NMT_tutorial/clean-data/

### basic tools
#
# moses decoder
decoder = $moses-bin-dir/moses

# conversion of phrase table into binary on-disk format
#ttable-binarizer = $moses-bin-dir/processPhraseTable
#ttable-binarizer = $moses-bin-dir/processPhraseTableMin

# conversion of rule table into binary on-disk format
#ttable-binarizer = "$moses-bin-dir/CreateOnDiskPt 1 1 4 100 2"

# tokenizers - comment out if all your data is already tokenized
#input-tokenizer = "$moses-script-dir/tokenizer/tokenizer.perl -a -l $input-
extension"
#output-tokenizer = "$moses-script-dir/tokenizer/tokenizer.perl -a -l $output-
extension"

# truecasers - comment out if you do not use the truecaser
#input-truecaser = $moses-script-dir/recaser/truecase.perl
#output-truecaser = $moses-script-dir/recaser/truecase.perl
#dettruecaser = $moses-script-dir/recaser/dettruecase.perl

```

```

### generic parallelizer for cluster and multi-core machines
# you may specify a script that allows the parallel execution
# parallizable steps (see meta file). you also need specify
# the number of jobs (cluster) or cores (multicore)
#
generic-parallelizer = $moses-script-dir/ems/support/generic-parallelizer.perl
#generic-parallelizer = $moses-script-dir/ems/support/generic-multicore-
parallelizer.perl

### cluster settings (if run on a cluster machine)
# number of jobs to be submitted in parallel
#
#jobs = 10

# arguments to qsub when scheduling a job
#qsub-settings = ""

# project for privileges and usage accounting
#qsub-project = iccs_smt

# memory and time
#qsub-memory = 4
#qsub-hours = 48

### multi-core settings
# when the generic parallelizer is used, the number of cores
# specified here
#cores = 8

#####
# PARALLEL CORPUS PREPARATION:
# create a tokenized, sentence-aligned corpus, ready for training

[CORPUS]

### long sentences are filtered out, since they slow down GIZA++
# and are a less reliable source of data. set here the maximum
# length of a sentence
#
max-sentence-length = 80

[CORPUS:thmy]

### command to run to get raw corpus files
#
# get-corpus-script =

### raw corpus files (untokenized, but sentence aligned)

```

```

#
raw-stem = $myrk-data/train

### tokenized corpus files (may contain long sentences)
#
#tokenized-stem = $myrk-data/train

### if sentence filtering should be skipped,
# point to the clean training data
#
#clean-stem =

### if corpus preparation should be skipped,
# point to the prepared training data
#
#lowercased-stem =

#####
# LANGUAGE MODEL TRAINING

[LM]

### tool to be used for language model training
# srilm
#lm-training = /data/lttools/training/srilm/srilm-1.6.0/bin/i686-m64/ngram-count
#lm-training = /home/ros/tool/srilm-1.7.1/bin/i686-m64/ngram-count
#settings = "-interpolate -kndiscount -unk"

# firstlm training
# msb = modified kneser ney; p=0 no singleton pruning
#lm-training = "$moses-script-dir/generic/trainlm-first2.perl -cores $cores
-first-dir $firstlm-dir -temp-dir $working-dir/tmp"
#settings = "-s msb -p 0"

# kenlm
# additional parameters to lmplz (check lmplz help message)
#settings = "-T $working-dir/tmp -S 10G"
# this tells EMS to use lmplz and tells EMS where lmplz is located
#lm-training = "$moses-script-dir/generic/trainlm-lmplz.perl -lmplz $lmplz"
#lm-training = "$moses-script-dir/ems/support/lmplz-wrapper.perl -bin $moses-
bin-dir/lmplz"

# kenlm training
lm-training = "$moses-script-dir/ems/support/lmplz-wrapper.perl -bin $moses-bin-
dir/lmplz"
settings = "--prune '0 0 1' -T $working-dir/lm -S 20% --discount_fallback"

```

```

#lm-binarizer = $moses-bin-dir/build_binary

# order of the language model
order = 5

### tool to be used for training randomized language model from scratch
# (more commonly, a SRILM is trained)
#
#rlm-training = "$randlm-dir/buildlm -falsepos 8 -values 8"

### script to use for binary table format for irstlm or kenlm
# (default: no binarization)

# irstlm
#lm-binarizer = $irstlm-dir/compile-lm

# kenlm, also set type to 8
lm-binarizer = $moses-bin-dir/build_binary
type = 8

### script to create quantized language model format (irstlm)
# (default: no quantization)
#
#lm-quantizer = $irstlm-dir/quantize-lm

### script to use for converting into randomized table format
# (default: no randomization)
#
#lm-randomizer = "$randlm-dir/buildlm -falsepos 8 -values 8"

### each language model to be used has its own section here

[LM:thmy]

### command to run to get raw corpus files
#
#get-corpus-script = ""

### raw corpus (untokenized)
#
raw-corpus = $myrk-data/train.$output-extension

### tokenized corpus files (may contain long sentences)
#
#tokenized-corpus = $my-data/train.$output-extension

### if corpus preparation should be skipped,
# point to the prepared language model

```

```

#
#lm =

#####
# INTERPOLATING LANGUAGE MODELS

[INTERPOLATED-LM]

# if multiple language models are used, these may be combined
# by optimizing perplexity on a tuning set
# see, for instance [Koehn and Schwenk, IJCNLP 2008]

### script to interpolate language models
# if commented out, no interpolation is performed
#
# script = $moses-script-dir/ems/support/interpolate-lm.perl

### tuning set
# you may use the same set that is used for mert tuning (reference set)
#
#tuning-sgm =
#raw-tuning =
#tokenized-tuning =
#factored-tuning =
#lowercased-tuning =
#split-tuning =

### group language models for hierarchical interpolation
# (flat interpolation is limited to 10 language models)
#group = "first,second fourth,fifth"

### script to use for binary table format for firstlm or kenlm
# (default: no binarization)

# firstlm
#lm-binarizer = $firstlm-dir/compile-lm

# kenlm, also set type to 8
lm-binarizer = $moses-bin-dir/build_binary
type = 8

### script to create quantized language model format (firstlm)
# (default: no quantization)
#
#lm-quantizer = $firstlm-dir/quantize-lm

### script to use for converting into randomized table format
# (default: no randomization)

```

```

#
#lm-randomizer = "$randlm-dir/buildlm -falsepos 8 -values 8"

#####
# MODIFIED MOORE LEWIS FILTERING

[MML] IGNORE

### specifications for language models to be trained
#
#lm-training = $srilm-dir/ngram-count
#lm-settings = "-interpolate -kndiscount -unk"
#lm-binarizer = $moses-src-dir/bin/build_binary
#lm-query = $moses-src-dir/bin/query
#order = 5

### in-/out-of-domain source/target corpora to train the 4 language model
#
# in-domain: point either to a parallel corpus
#outdomain-stem = [CORPUS:toy:clean-split-stem]

# ... or to two separate monolingual corpora
#indomain-target = [LM:toy:lowercased-corpus]
#raw-indomain-source = $toy-data/nc-5k.$input-extension

# point to out-of-domain parallel corpus
#outdomain-stem = [CORPUS:giga:clean-split-stem]

# settings: number of lines sampled from the corpora to train each language
model on
# (if used at all, should be small as a percentage of corpus)
#settings = "--line-count 100000"

#####
# TRANSLATION MODEL TRAINING

[TRAINING]

### training script to be used: either a legacy script or
# current moses training script (default)
#
script = $moses-script-dir/training/train-model.perl

### general options
# these are options that are passed on to train-model.perl, for instance
# * "-mgiza -mgiza-cpus 8" to use mgiza instead of giza
# * "-sort-buffer-size 8G -sort-compress gzip" to reduce on-disk sorting
# * "-sort-parallel 8 -cores 8" to speed up phrase table building

```

```

#
#training-options = "--sort-parallel 8 -cores 8"

#training-options = "-cores 8"

### factored training: specify here which factors used
# if none specified, single factor training is assumed
# (one translation step, surface to surface)
#
#input-factors = word lemma pos morph
#output-factors = word lemma pos
#alignment-factors = "word -> word"
#translation-factors = "word -> word"
#reordering-factors = "word -> word"
#generation-factors = "word -> pos"
#decoding-steps = "t0, g0"

### parallelization of data preparation step
# the two directions of the data preparation can be run in parallel
# comment out if not needed
#
parallel = yes

### pre-computation for giza++
# giza++ has a more efficient data structure that needs to be
# initialized with snt2cooc. if run in parallel, this may reduces
# memory requirements. set here the number of parts
#
#run-giza-in-parts = 5

### symmetrization method to obtain word alignments from giza output
# (commonly used: grow-diag-final-and)
#
alignment-symmetrization-method = grow-diag-final-and

### use of Chris Dyer's fast align for word alignment
#
#fast-align-settings = "-d -o -v"

### use of berkeley aligner for word alignment
#
#use-berkeley = true
#alignment-symmetrization-method = berkeley
#berkeley-train = $moses-script-dir/ems/support/berkeley-train.sh
#berkeley-process = $moses-script-dir/ems/support/berkeley-process.sh
#berkeley-jar = /your/path/to/berkeleyaligner-1.1/berkeleyaligner.jar
#berkeley-java-options = "-server -mx30000m -ea"
#berkeley-training-options = "-Main.iters 5 5 -EMWordAligner.numThreads 8"

```

```

#berkeley-process-options = "-EMWordAligner.numThreads 8"
#berkeley-posterior = 0.5

### use of baseline alignment model (incremental training)
#
#baseline = 68
#baseline-alignment-model = "$working-dir/training/prepared.$baseline/$input-
extension.vcb \
# $working-dir/training/prepared.$baseline/$output-extension.vcb \
# $working-dir/training/giza.$baseline/${output-extension}-$input-
extension.cooc \
# $working-dir/training/giza-inverse.$baseline/${input-extension}-$output-
extension.cooc \
# $working-dir/training/giza.$baseline/${output-extension}-$input-
extension.thmm.5 \
# $working-dir/training/giza.$baseline/${output-extension}-$input-
extension.hhmm.5 \
# $working-dir/training/giza-inverse.$baseline/${input-extension}-$output-
extension.thmm.5 \
# $working-dir/training/giza-inverse.$baseline/${input-extension}-$output-
extension.hhmm.5"

### if word alignment should be skipped,
# point to word alignment files
#
#word-alignment = $working-dir/model/aligned.1

### filtering some corpora with modified Moore-Lewis
# specify corpora to be filtered and ratio to be kept, either before or after
word alignment
#mml-filter-corpora = toy
#mml-before-wa = "-proportion 0.9"
#mml-after-wa = "-proportion 0.9"

### create a bilingual concordancer for the model
#
#biconcor = $moses-script-dir/ems/biconcor/biconcor

#####

### use of Operation Sequence Model
### Durrani, Schmid and Fraser. (2011): "A Joint Sequence Translation Model with
Integrated Reordering"

#operation-sequence-model = "yes"
#operation-sequence-model-order = 5
#operation-sequence-model-settings = ""

```

```

### compile Moses with --max-kenlm-order=9 if higher order is required

### if OSM training should be skipped,
# point to OSM Model
#
# osm-model =

#####

### lexicalized reordering: specify orientation type
# (default: only distance-based reordering model)
#
lexicalized-reordering = msd-bidirectional-fe

### hierarchical rule set
#
#hierarchical-rule-set = true

### settings for rule extraction
#
#extract-settings = ""
max-phrase-length = 5

### add extracted phrases from baseline model
#
#baseline-extract = $working-dir/model/extract.$baseline
#
# requires aligned parallel corpus for re-estimating lexical translation
probabilities
#baseline-corpus = $working-dir/training/corpus.$baseline
#baseline-alignment = $working-dir/model/aligned.$baseline.$alignment-
symmetrization-method

### unknown word labels (target syntax only)
# enables use of unknown word labels during decoding
# label file is generated during rule extraction
#
#use-unknown-word-labels = true

### if phrase extraction should be skipped,
# point to stem for extract files
#
# extracted-phrases =

### settings for rule scoring
#
score-settings = "--GoodTuring"

```

```

### include word alignment in phrase table
#
#include-word-alignment-in-rules = yes

### sparse lexical features
#
#sparse-features = "target-word-insertion top 50, source-word-deletion top 50,
word-translation top 50 50, phrase-length"

### domain adaptation settings
# options: sparse, any of: indicator, subset, ratio
#domain-features = "subset"

### if phrase table training should be skipped,
# point to phrase translation table
#
# phrase-translation-table =

### if reordering table training should be skipped,
# point to reordering table
#
# reordering-table =

### filtering the phrase table based on significance tests
# Johnson, Martin, Foster and Kuhn. (2007): "Improving Translation Quality by
Discarding Most of the Phrasetable"
# options: -n number of translations; -l 'a+e', 'a-e', or a positive real value
-log prob threshold
#salm-index = /path/to/project/salm/Bin/Linux/Index/IndexSA.064
#sigtest-filter = "-l a+e -n 50"

### if training should be skipped,
# point to a configuration file that contains
# pointers to all relevant model files
#
#config-with-reused-weights =

#####
### TUNING: finding good weights for model components

[TUNING]

### instead of tuning with this setting, old weights may be recycled
# specify here an old configuration file with matching weights
#
#weight-config = /panfs/panltg2/users/finch/expt/moses2.1/plain.weight.ini

```

```

### tuning script to be used
#
tuning-script = $moses-script-dir/training/mert-moses.pl
tuning-settings = "-mertdir $moses-bin-dir"

### specify the corpus used for tuning
# it should contain 1000s of sentences
#
#input-smg =
raw-input = $myrk-data/dev.$input-extension

#tokenized-input =
#factorized-input =
#input =
#
#reference-smg =
raw-reference = $myrk-data/dev.$output-extension

#tokenized-reference =
#factorized-reference =
#reference =

### size of n-best list used (typically 100)
#
nbest = 100

### ranges for weights for random initialization
# if not specified, the tuning script will use generic ranges
# it is not clear, if this matters
#
# lambda =

### additional flags for the filter script
#
filter-settings = ""

### additional flags for the decoder
#
decoder-settings = ""

### if tuning should be skipped, specify this here
# and also point to a configuration file that contains
# pointers to all relevant model files
#
#config =

#####
## RECASER: restore case, this part only trains the model

```

[RECASING]

```
#decoder = $moses-bin-dir/moses

### training data
# raw input needs to be still tokenized,
# also also tokenized input may be specified
#
#tokenized = [LM:europarl:tokenized-corpus]

# recase-config =

#lm-training = $srilm-dir/ngram-count

#####
## TRUECASER: train model to truecase corpora and input
```

[TRUECASER]

```
### script to train truecaser models
#
#trainer = $moses-script-dir/recaser/train-truecaser.perl

### training data
# data on which truecaser is trained
# if no training data is specified, parallel corpus is used
#
# raw-stem =
# tokenized-stem =

### trained model
#
# truecase-model =

#####
## EVALUATION: translating a test set using the tuned system and score it
```

[EVALUATION]

```
### additional flags for the filter script
#
#filter-settings = ""

### additional decoder settings
# switches for the Moses decoder
# common choices:
#   "-threads N" for multi-threading
```

```

# "-mbr" for MBR decoding
# "-drop-unknown" for dropping unknown source words
# "-search-algorithm 1 -cube-pruning-pop-limit 5000 -s 5000" for cube pruning
#
decoder-settings = "-search-algorithm 1 -cube-pruning-pop-limit 5000 -s 5000"

### specify size of n-best list, if produced
#
#nbest = 100

### multiple reference translations
#
#multiref = yes

### prepare system output for scoring
# this may include detokenization and wrapping output in sgm
# (needed for nist-bleu, ter, meteor)
#
#detokenizer = "$moses-script-dir/tokenizer/detokenizer.perl -l $output-
extension"
#recaser = $moses-script-dir/recaser/recase.perl
wrapping-script = "$moses-script-dir/ems/support/wrap-xml.perl $output-
extension"
#output-sgm =

### BLEU
#
nist-bleu = $moses-script-dir/generic/mteval-v13a.pl
nist-bleu-c = "$moses-script-dir/generic/mteval-v13a.pl -c"
#multi-bleu = $moses-script-dir/generic/multi-bleu.perl
#ibm-bleu =

### TER: translation error rate (BBN metric) based on edit distance
# not yet integrated
#
# ter =

### METEOR: gives credit to stem / worknet synonym matches
# not yet integrated
#
# meteor =

### Analysis: carry out various forms of analysis on the output
#
analysis = $moses-script-dir/ems/support/analysis.perl
#
# also report on input coverage
analyze-coverage = yes

```

```

#
# also report on phrase mappings used
report-segmentation = yes
#
# report precision of translations for each input word, broken down by
# count of input word in corpus and model
report-precision-by-coverage = yes
#
# further precision breakdown by factor
#precision-by-coverage-factor = pos
#
# visualization of the search graph in tree-based models
#analyze-search-graph = yes

[EVALUATION:test]

### input data
#
#input-smg = /panfs/panmt/users/ye/cluster/test-smg/test.$input-
extension.src.sgm
input-smg = $myrk-data/test-smg/test.$input-extension.src.sgm
#input-smg = $mykc-data/test-smg/test.$input-extension.src.sgm

# raw-input = $mykc-data/test.$input-extension
# tokenized-input =
# factorized-input =
# input =

### reference data
#
#reference-smg = /panfs/panmt/users/ye/cluster/test-smg/test.$output-
extension.ref.sgm
reference-smg = $myrk-data/test-smg/test.$output-extension.ref.sgm
#reference-smg = $mykc-data/test-smg/test.$output-extension.ref.sgm

# raw-reference = $mykc-data/test.$output-extension
# tokenized-reference =
# reference =

### analysis settings
# may contain any of the general evaluation analysis settings
# specific setting: base coverage statistics on earlier run
#
#precision-by-coverage-base = $working-dir/evaluation/test.analysis.5

### wrapping frame
# for nist-bleu and other scoring scripts, the output needs to be wrapped
# in smg markup (typically like the input smg)

```

```
#
wrapping-frame = $input-smg

#####
### REPORTING: summarize evaluation scores

[REPORTING]

### currently no parameters for reporting section
```

1.11 show updated perl

1.12 Let's DO SMT

```
[176]: %pwd
```

```
[176]: '/home/ye/exp/SMT-NMT_tutorial/pbsmt'
```

```
[178]: !time perl ./run-baseline.pl
```

```
my-th-baseline /home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/my-
th/config.baseline.my-th
th-my-baseline /home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/th-
my/config.baseline.th-my
```

```
real    0m0.024s
user    0m0.019s
sys     0m0.006s
```

1.13 ERROR

```
[179]: !cat ./run1.log
```

```
sh: 1: /home/lar/tool/moses/scripts/ems/experiment.perl: not found
sh: 1: /home/lar/tool/moses/scripts/ems/experiment.perl: not found
sh: 1: /home/lar/tool/moses/scripts/ems/experiment.perl: not found
sh: 1: /home/lar/tool/moses/scripts/ems/experiment.perl: not found
```

1.14 Found Error

```
run-baseline.pl
```

```
chomp $config;
```

```
my @toks = split /\./, $config;
my $jobName = "$toks[$#toks]-$toks[$#toks - 1]";
```

```
print "$jobName $config\n";
```

```
`/home/lar/tool/moses/scripts/ems/experiment.perl -config $config -exec -no-graph 2>&1 | t
```

```
#~/home/ros/mosesdecoder/scripts/ems/experiment.perl -config $config -exec -no-graph`;
```

1.15 Run Again

```
[180]: %pwd
```

```
[180]: '/home/ye/exp/SMT-NMT_tutorial/pbsmt'
```

```
[181]: !time perl ./run-baseline.pl
```

```
my-th-baseline /home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/my-  
th/config.baseline.my-th  
th-my-baseline /home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/th-  
my/config.baseline.th-my
```

```
real    0m0.139s  
user    0m0.106s  
sys     0m0.026s
```

1.16 We Forgot!

```
[184]: %cd /home/ye/exp/SMT-NMT_tutorial/pbsmt
```

```
/home/ye/exp/SMT-NMT_tutorial/pbsmt
```

```
[186]: %pwd
```

```
[186]: '/home/ye/exp/SMT-NMT_tutorial/pbsmt'
```

```
[185]: !perl ./generate_configs.pl
```

```
Can't open perl script "./generate_configs.pl": No such file or directory
```

```
[187]: !time perl ./run-baseline.pl
```

```
my-th-baseline /home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/my-  
th/config.baseline.my-th  
th-my-baseline /home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/th-  
my/config.baseline.th-my
```

```
real    0m0.107s  
user    0m0.083s  
sys     0m0.028s
```

1.17 One More Error Example

```
[188]: %cd /home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/th-my/steps/1
```

```
/home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/th-my/steps/1
```

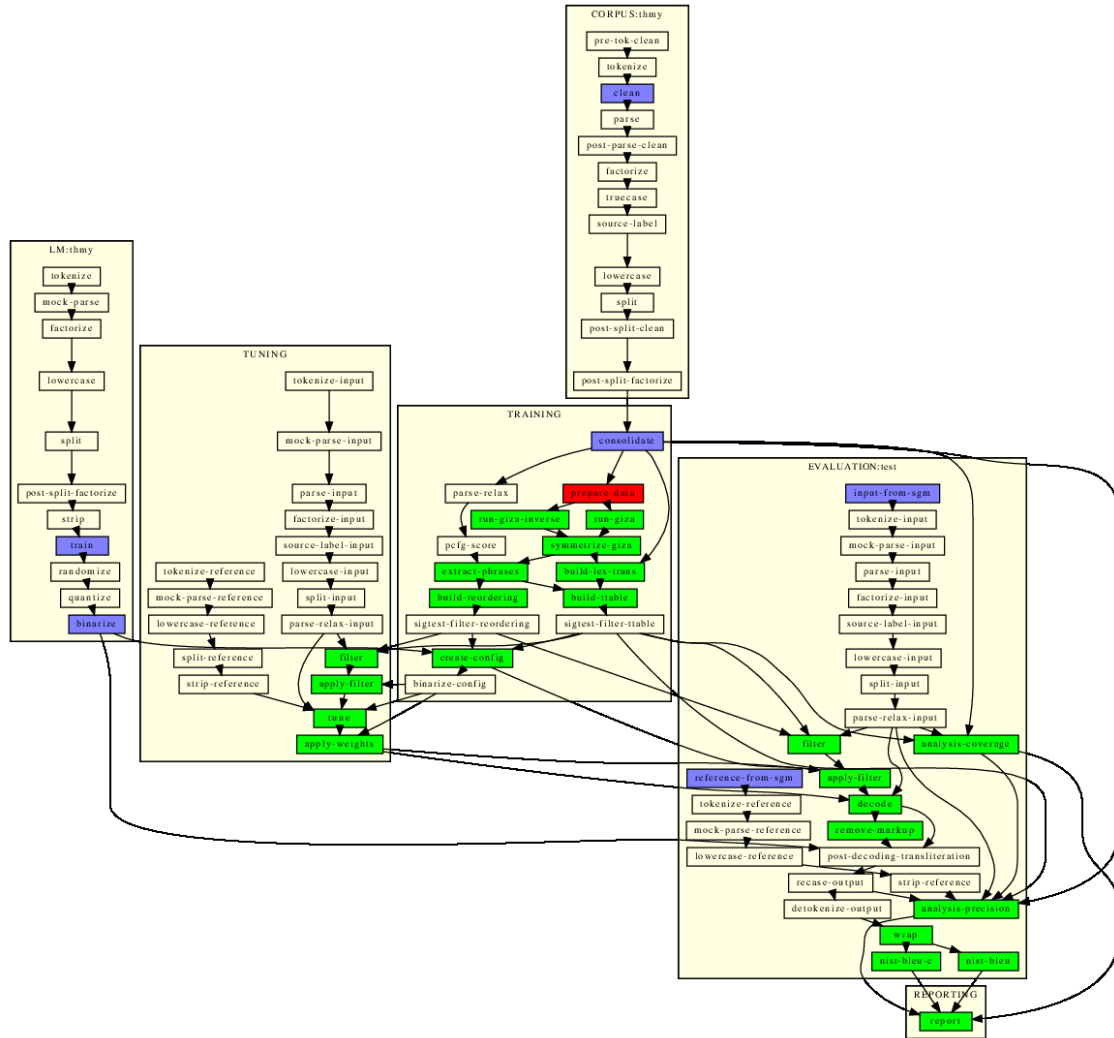
[189]: !cat TRAINING_prepare-data.1.STDERR

```
Using SCRIPTS_ROOTDIR: /home/ye/tool/mosesbin/ubuntu-17.04/moses/scripts
Using single-thread GIZA
using pigz
(1) preparing corpus @ Sun Apr 26 15:46:43 +07 2026
Executing: mkdir -p /home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/th-
my/training/prepared.1
(1.0) selecting factors @ Sun Apr 26 15:46:43 +07 2026
(1.1) running mkcls @ Sun Apr 26 15:46:43 +07 2026
/home/ye/tool/giza-pp/GIZA++-v2/mkcls -c50 -n2 -p/home/ye/exp/SMT-
NMT_tutorial/pbsmt/baseline/th-my/training/corpus.1.th -V/home/ye/exp/SMT-
NMT_tutorial/pbsmt/baseline/th-my/training/prepared.1/th.vcb.classes opt
(1.2) creating vcb file /home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/th-
my/training/prepared.1/th.vcb @ Sun Apr 26 15:46:43 +07 2026
Executing: /home/ye/tool/giza-pp/GIZA++-v2/mkcls -c50 -n2 -p/home/ye/exp/SMT-
NMT_tutorial/pbsmt/baseline/th-my/training/corpus.1.th -V/home/ye/exp/SMT-
NMT_tutorial/pbsmt/baseline/th-my/training/prepared.1/th.vcb.classes opt
(1.1) running mkcls @ Sun Apr 26 15:46:43 +07 2026
/home/ye/tool/giza-pp/GIZA++-v2/mkcls -c50 -n2 -p/home/ye/exp/SMT-
NMT_tutorial/pbsmt/baseline/th-my/training/corpus.1.my -V/home/ye/exp/SMT-
NMT_tutorial/pbsmt/baseline/th-my/training/prepared.1/my.vcb.classes opt
Executing: /home/ye/tool/giza-pp/GIZA++-v2/mkcls -c50 -n2 -p/home/ye/exp/SMT-
NMT_tutorial/pbsmt/baseline/th-my/training/corpus.1.my -V/home/ye/exp/SMT-
NMT_tutorial/pbsmt/baseline/th-my/training/prepared.1/my.vcb.classes opt
Can't exec "/home/ye/tool/giza-pp/GIZA++-v2/mkcls": No such file or directory at
/home/ye/tool/mosesbin/ubuntu-17.04/moses/scripts/training/train-model.perl line
2435.
Can't exec "/home/ye/tool/giza-pp/GIZA++-v2/mkcls": No such file or directory at
/home/ye/tool/mosesbin/ubuntu-17.04/moses/scripts/training/train-model.perl line
2435.
ERROR: Failed to execute: /home/ye/tool/giza-pp/GIZA++-v2/mkcls -c50 -n2
-p/home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/th-my/training/corpus.1.th
-V/home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/th-
my/training/prepared.1/th.vcb.classes opt
No such file or directory
ERROR: Failed to execute: /home/ye/tool/giza-pp/GIZA++-v2/mkcls -c50 -n2
-p/home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/th-my/training/corpus.1.my
-V/home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/th-
my/training/prepared.1/my.vcb.classes opt
No such file or directory
(1.2) creating vcb file /home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/th-
my/training/prepared.1/my.vcb @ Sun Apr 26 15:46:43 +07 2026
(1.3) numberizing corpus /home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/th-
my/training/prepared.1/th-my-int-train.snt @ Sun Apr 26 15:46:43 +07 2026
(1.3) numberizing corpus /home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/th-
my/training/prepared.1/my-th-int-train.snt @ Sun Apr 26 15:46:43 +07 2026
```

1.18 To Add graph1.png

```
[1]: from IPython.display import Image
Image(filename='/home/ye/exp/SMT-NMT_tutorial/error-examples/GIZA-error.png',
      width=800)
```

[1]:



1.19 Solution

အထက်ပါ error ကိုတော့ အတွေ့အကြုံအရ သိနေတာမို့ mkcls binary ဖိုင်ကို GIZA++-v2/ ဖိုလ်ဒါအောက်ကို ကော်ပီကူးထည့်ပေးလိုက်ယုံနဲ့ ပြေလည်ပါတယ်။ အောက်ပါအတိုင်းပါ။

```
ye@lst-hpc3090:~/tool/giza-pp$ cp ./mkcls-v2/mkcls ./GIZA++-v2/
```

1.20 ERROR

တချို့ error တွေက terminal မှာ ဘာမှမပြပဲ ရပ်သွားတာမျိုးလည်း ဖြစ်တတ်ပါတယ်။ အဲဒါဆိုရင်

အများသောအားဖြင့် perl script မှာ path ပေးတာ လွဲသွားလို့ပါ။

ဘာဖိုင်မှ ထွက်မလာတာကို တမင်တကာ လုပ်ပြထားတာပါ။

config နဲ့ SMT ကို run မယ်ဆိုရင် နောက်ထပ် ပြင်ရမယ့် အဆင့် ရှိသေးတယ်။ အဲဒါက ဒေတာဖိုင်ဒါအောက်မှာ ဝင်လုပ်ရမယ်။

Refer following link:

<https://github.com/ye-kyaw-thu/MTRSS/tree/master/pbsmt/data>

ဒီတစ်ခေါက် tutorial အတွက်တော့ SMT-NMT_tutorial/pbsmt/ ဖိုင်ဒါအောက်မှာလည်း ကောပီကူးထည့်ပေးထားမယ်။

အဲဒီ script ကို ကိုယ့် စက်ထဲက parallel data ဒေတာ ဖိုင်ဒါအောက်ကို ကောပီကူးယူပြီး ပြင်ဆင်ကြပါ။

```
[125]: %cd ../../
```

```
/home/ye/exp/SMT-NMT_tutorial
```

```
[127]: !ls ./scripts
```

```
generate_sgms.pl  ref2sgm.pl  src2sgm.pl
```

```
[128]: cp -r scripts/ ./clean-data/
```

path ပြင်ပြီးသား update လုပ်ပြီးသား perl script တွေနဲ့ run ကြရလိမ့်မယ်။ ဥပမာ အောက်ပါအတိုင်း

```
[131]: %cd /home/ye/exp/SMT-NMT_tutorial/clean-data/
```

```
/home/ye/exp/SMT-NMT_tutorial/clean-data
```

```
[132]: %cd scripts/
```

```
/home/ye/exp/SMT-NMT_tutorial/clean-data/scripts
```

```
[134]: !tree .
```

```
.
├── generate_sgms.pl
├── ref2sgm.pl
└── src2sgm.pl
```

```
1 directory, 3 files
```

```
[135]: !perl ./generate_sgms.pl
```

```
sh: 1: ./ref2sgm.pl: Permission denied
```

```
sh: 1: ./src2sgm.pl: Permission denied
```

run-pbsmt.sh ကို Executable ဖြစ်အောင် permission ပေးကြရအောင်...

```
[109]: !chmod +x ./run-pbsmt.sh
```

```
[110]: !cat run-pbsmt.sh
```

```
#!/bin/bash

# Written by Ye, NICT, Kyoto, Japan
# How to run: time bash ./run-pbsmt.sh

# after running this script, you will get baseline/ folder
perl ./generate_configs.pl

# start running PBSMT
nohup time perl ./run-baseline.pl
#perl ./run-baseline.pl
```

Shell script ကို run မယ်

```
[111]: !./run-pbsmt.sh
```

```
Can't open perl script "./generate_configs.pl": No such file or directory
nohup: ignoring input and appending output to 'nohup.out'
```

1.21 Notes

ဒီ notebook မှာ ကြုံခဲ့ရသလိုပါပဲ error အများစုက path လွဲတာ၊ config ဖိုင်မှာ တခုခု update လုပ်ဖို့ ကျန်တာ၊ test data တွေကို SGM format ပြောင်းဖို့ မေ့ကျန်ခဲ့တာ၊ GIZA++ ရဲ့ mkcls ကို ရှာမတွေ့တာ တွေပါပဲ။ အဲဒီလိုမျိုး error မှ မဟုတ်ရင်တော့ ဒေတာနဲ့ ဆိုင်ပါလိမ့်မယ်။ Parallel corpus မှာ တချို့ blank line တွေ ပါနေတာတို့ special character တွေကို ရှင်းဖို့ လိုနေသေးတာတို့ ဖြစ်ဖို့ များပါတယ်။

Error အကုန် ရှင်းပြီးရင်တော့ run လိုက်ရင် အစအဆုံး ပြီးအောင် run ပေးသွားပါလိမ့်မယ်။

1.22 PBSMT Experiment

```
[2]: %pwd
```

```
[2]: '/home/ye/exp/SMT-NMT_tutorial'
```

```
[3]: %cd ./pbsmt/
```

```
/home/ye/exp/SMT-NMT_tutorial/pbsmt
```

```
[4]: !ls *
```

```
config.baseline      nohup.out  run-baseline.pl
generate_configs.pl  run1.log   run-pbsmt.sh
```

```
[5]: !perl ./generate_configs.pl
```

```
[7]: !ls * --color=auto
```

```
config.baseline      nohup.out  run-baseline.pl
generate_configs.pl  run1.log   run-pbsmt.sh
```

```
baseline:
my-th th-my
```

```
[8]: !tree ./baseline
```

```
./baseline
  my-th
    config.baseline.my-th
  th-my
    config.baseline.th-my
```

```
3 directories, 2 files
```

```
[9]: !time perl run-baseline.pl
```

```
my-th-baseline /home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/my-
th/config.baseline.my-th
th-my-baseline /home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/th-
my/config.baseline.th-my
```

```
real    20m7.036s
user    0m8.171s
sys     0m3.211s
```

Training data က စာကြောင်းရေ ၅၀၀၀ ကျော်၊ development data က စာကြောင်းရေ ၅၀၀ ကျော်နဲ့ test data က စာကြောင်းရေ ၁၀၀ နဲ့ပဲ run တာမို့လို့ မြန်မာ-ထိုင်း၊ ထိုင်း-မြန်မာ အတွက်က ဆာဗာပေါ်မှာ မိနစ် ၂၀ကျော်ပဲ ကြာပါတယ်။

System တစ်ခုလုံး run သွားတဲ့ ဂရပ်တွေကို ကြည့်ကြည့်ရအောင်။

အောက်ပါ ဂရပ်က မြန်မာ-ထိုင်း အတွက်ပါ။

```
[10]: %cd /home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/my-th/steps/1
```

```
/home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/my-th/steps/1
```

```
[11]: !ls *
```

```
config.1
CORPUS_thmy_clean.1
CORPUS_thmy_clean.1.DONE
CORPUS_thmy_clean.1.INFO
CORPUS_thmy_clean.1.STDERR
CORPUS_thmy_clean.1.STDERR.digest
CORPUS_thmy_clean.1.STDOUT
EVALUATION_test_analysis-coverage.1
EVALUATION_test_analysis-coverage.1.DONE
EVALUATION_test_analysis-coverage.1.INFO
EVALUATION_test_analysis-coverage.1.STDERR
EVALUATION_test_analysis-coverage.1.STDERR.digest
```

EVALUATION_test_analysis-coverage.1.STDOUT
EVALUATION_test_analysis-precision.1
EVALUATION_test_analysis-precision.1.DONE
EVALUATION_test_analysis-precision.1.INFO
EVALUATION_test_analysis-precision.1.STDERR
EVALUATION_test_analysis-precision.1.STDERR.digest
EVALUATION_test_analysis-precision.1.STDOUT
EVALUATION_test_apply-filter.1
EVALUATION_test_apply-filter.1.DONE
EVALUATION_test_apply-filter.1.INFO
EVALUATION_test_apply-filter.1.STDERR
EVALUATION_test_apply-filter.1.STDERR.digest
EVALUATION_test_apply-filter.1.STDOUT
EVALUATION_test_decode.1
EVALUATION_test_decode.1.DONE
EVALUATION_test_decode.1.INFO
EVALUATION_test_decode.1.STDERR
EVALUATION_test_decode.1.STDERR.digest
EVALUATION_test_decode.1.STDOUT
EVALUATION_test_filter.1
EVALUATION_test_filter.1.DONE
EVALUATION_test_filter.1.INFO
EVALUATION_test_filter.1.STDERR
EVALUATION_test_filter.1.STDERR.digest
EVALUATION_test_filter.1.STDOUT
EVALUATION_test_input-from-sgm.1
EVALUATION_test_input-from-sgm.1.DONE
EVALUATION_test_input-from-sgm.1.INFO
EVALUATION_test_input-from-sgm.1.STDERR
EVALUATION_test_input-from-sgm.1.STDERR.digest
EVALUATION_test_input-from-sgm.1.STDOUT
EVALUATION_test_nist-bleu.1
EVALUATION_test_nist-bleu.1.DONE
EVALUATION_test_nist-bleu.1.INFO
EVALUATION_test_nist-bleu.1.STDERR
EVALUATION_test_nist-bleu.1.STDERR.digest
EVALUATION_test_nist-bleu.1.STDOUT
EVALUATION_test_nist-bleu-c.1
EVALUATION_test_nist-bleu-c.1.DONE
EVALUATION_test_nist-bleu-c.1.INFO
EVALUATION_test_nist-bleu-c.1.STDERR
EVALUATION_test_nist-bleu-c.1.STDERR.digest
EVALUATION_test_nist-bleu-c.1.STDOUT
EVALUATION_test_reference-from-sgm.1
EVALUATION_test_reference-from-sgm.1.DONE
EVALUATION_test_reference-from-sgm.1.INFO
EVALUATION_test_reference-from-sgm.1.STDERR
EVALUATION_test_reference-from-sgm.1.STDERR.digest

EVALUATION_test_reference-from-sgm.1.STDOUT
EVALUATION_test_remove-markup.1
EVALUATION_test_remove-markup.1.DONE
EVALUATION_test_remove-markup.1.INFO
EVALUATION_test_remove-markup.1.STDERR
EVALUATION_test_remove-markup.1.STDERR.digest
EVALUATION_test_remove-markup.1.STDOUT
EVALUATION_test_wrap.1
EVALUATION_test_wrap.1.DONE
EVALUATION_test_wrap.1.INFO
EVALUATION_test_wrap.1.STDERR
EVALUATION_test_wrap.1.STDERR.digest
EVALUATION_test_wrap.1.STDOUT
graph.1.dot
graph.1.png
graph.1.ps
LM_thmy_binarize.1
LM_thmy_binarize.1.DONE
LM_thmy_binarize.1.INFO
LM_thmy_binarize.1.STDERR
LM_thmy_binarize.1.STDERR.digest
LM_thmy_binarize.1.STDOUT
LM_thmy_train.1
LM_thmy_train.1.DONE
LM_thmy_train.1.INFO
LM_thmy_train.1.STDERR
LM_thmy_train.1.STDERR.digest
LM_thmy_train.1.STDOUT
parameter.1
re-use.1
running.1
TRAINING_build-lex-trans.1
TRAINING_build-lex-trans.1.DONE
TRAINING_build-lex-trans.1.INFO
TRAINING_build-lex-trans.1.STDERR
TRAINING_build-lex-trans.1.STDERR.digest
TRAINING_build-lex-trans.1.STDOUT
TRAINING_build-reordering.1
TRAINING_build-reordering.1.DONE
TRAINING_build-reordering.1.INFO
TRAINING_build-reordering.1.STDERR
TRAINING_build-reordering.1.STDERR.digest
TRAINING_build-reordering.1.STDOUT
TRAINING_build-ttable.1
TRAINING_build-ttable.1.DONE
TRAINING_build-ttable.1.INFO
TRAINING_build-ttable.1.STDERR
TRAINING_build-ttable.1.STDERR.digest

TRAINING_build-ttable.1.STDOUT
TRAINING_consolidate.1
TRAINING_consolidate.1.DONE
TRAINING_consolidate.1.INFO
TRAINING_consolidate.1.STDERR
TRAINING_consolidate.1.STDERR.digest
TRAINING_consolidate.1.STDOUT
TRAINING_create-config.1
TRAINING_create-config.1.DONE
TRAINING_create-config.1.INFO
TRAINING_create-config.1.STDERR
TRAINING_create-config.1.STDERR.digest
TRAINING_create-config.1.STDOUT
TRAINING_extract-phrases.1
TRAINING_extract-phrases.1.DONE
TRAINING_extract-phrases.1.INFO
TRAINING_extract-phrases.1.STDERR
TRAINING_extract-phrases.1.STDERR.digest
TRAINING_extract-phrases.1.STDOUT
TRAINING_prepare-data.1
TRAINING_prepare-data.1.DONE
TRAINING_prepare-data.1.INFO
TRAINING_prepare-data.1.STDERR
TRAINING_prepare-data.1.STDERR.digest
TRAINING_prepare-data.1.STDOUT
TRAINING_run-giza.1
TRAINING_run-giza.1.DONE
TRAINING_run-giza.1.INFO
TRAINING_run-giza.1.STDERR
TRAINING_run-giza.1.STDERR.digest
TRAINING_run-giza.1.STDOUT
TRAINING_run-giza-inverse.1
TRAINING_run-giza-inverse.1.DONE
TRAINING_run-giza-inverse.1.INFO
TRAINING_run-giza-inverse.1.STDERR
TRAINING_run-giza-inverse.1.STDERR.digest
TRAINING_run-giza-inverse.1.STDOUT
TRAINING_symmetrize-giza.1
TRAINING_symmetrize-giza.1.DONE
TRAINING_symmetrize-giza.1.INFO
TRAINING_symmetrize-giza.1.STDERR
TRAINING_symmetrize-giza.1.STDERR.digest
TRAINING_symmetrize-giza.1.STDOUT
TUNING_apply-filter.1
TUNING_apply-filter.1.DONE
TUNING_apply-filter.1.INFO
TUNING_apply-filter.1.STDERR
TUNING_apply-filter.1.STDERR.digest

```

TUNING_apply-filter.1.STDOUT
TUNING_apply-weights.1
TUNING_apply-weights.1.DONE
TUNING_apply-weights.1.INFO
TUNING_apply-weights.1.STDERR
TUNING_apply-weights.1.STDERR.digest
TUNING_apply-weights.1.STDOUT
TUNING_filter.1
TUNING_filter.1.DONE
TUNING_filter.1.INFO
TUNING_filter.1.STDERR
TUNING_filter.1.STDERR.digest
TUNING_filter.1.STDOUT
TUNING_tune.1
TUNING_tune.1.DONE
TUNING_tune.1.INFO
TUNING_tune.1.STDERR
TUNING_tune.1.STDERR.digest
TUNING_tune.1.STDOUT

```

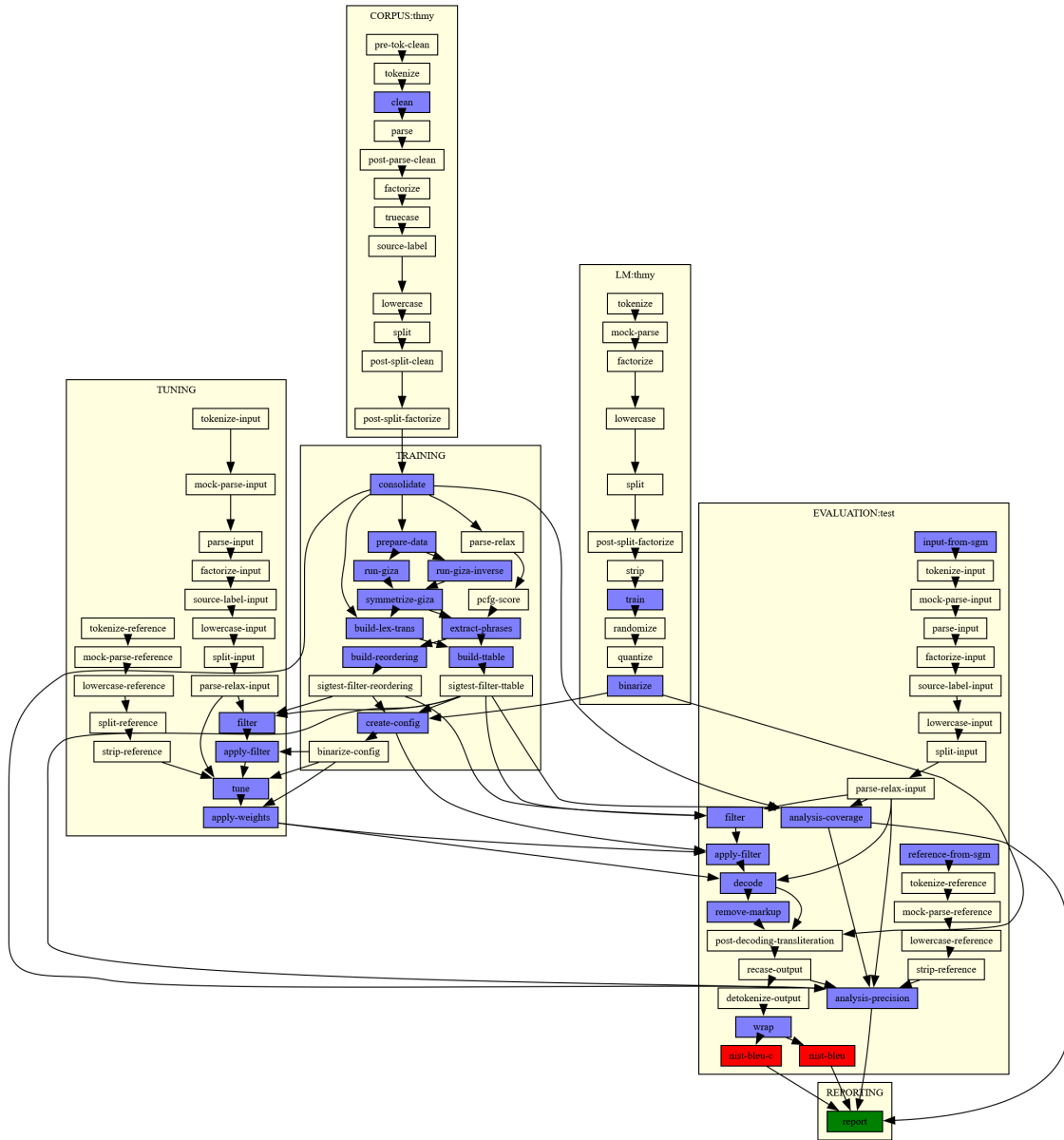
```
[12]: !ls *.png
```

```
graph.1.png
```

graph.1.png ကိုပဲ quality ကောင်းအောင် ပုံပြင်ထုတ်ထားပြီး error-examples/ ဖိုလ်ဒါအောက်ထဲမှာပဲ သိမ်းထားပေးလိုက်ပါတယ်။

```
[13]: from IPython.display import Image
      Image(filename='/home/ye/exp/SMT-NMT_tutorial/error-examples/graph.1.my-th.
      ↳png', width=800)
```

```
[13]:
```



အောက်ဆုံးပိုင်းမှာ အနီနဲ့ nist-bleu score နဲ့ ပတ်သက်ပြီး ပြနေပေမဲ့ စိုးရိမ်စရာ မရှိပါဘူး။ ဒါပေမဲ့ multi-bleu နဲ့ တွက်ထားတာကိုတော့ ဆရာတို့ လိုချင်တယ်။ အဲဒါက ပါမလာဘူး။ config မှိုင်းထဲမှာ assign မလုပ်ထားလို့ ပေးနေတဲ့ error လို့ ထင်ပါတယ်။

Config မှိုင်း template ကို ဝင်ကြည့်တော့ အောက်ပါအတိုင်း comment ပိတ်ထားတာကို တွေ့ရတယ်။ သေချာတယ်။ ကျောင်းသားတွေကို ကိုယ်ဖာသာကိုယ် ရှာတွက်တတ်ဖို့ exercise လုပ်ဖို့ ပိတ်ထားခဲ့တာ။

BLEU

#

nist-bleu = \$moses-script-dir/generic/mteval-v13a.pl

nist-bleu-c = "\$moses-script-dir/generic/mteval-v13a.pl -c"

```
#multi-bleu = $moses-script-dir/generic/multi-bleu.perl
#ibm-bleu =
```

အထက်က multi-bleu လိုင်းကိုပါ ဖွင့်ပေးပြီး နောက်တစ်ခေါက် ထပ် run ကြည့်ရအောင်။

```
[17]: %cd /home/ye/exp/SMT-NMT_tutorial/pbsmt
```

```
/home/ye/exp/SMT-NMT_tutorial/pbsmt
```

```
[18]: %rm -r baseline/
```

```
[20]: !perl ./generate_configs.pl
```

```
[22]: !tree ./baseline
```

```
./baseline
  my-th
    config.baseline.my-th
  th-my
    config.baseline.th-my
```

3 directories, 2 files

config မှိုင်းကို ပြင်ပြီး နောက်တစ်ခေါက် ထပ် run မယ်။

```
[23]: !time perl ./run-baseline.pl
```

```
my-th-baseline /home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/my-
th/config.baseline.my-th
th-my-baseline /home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/th-
my/config.baseline.th-my
```

```
real    11m10.345s
user    0m8.018s
sys     0m2.715s
```

test-run လုပ်ထားတဲ့ my-th ရလဒ်ကို ဝင်လေ့လာရအောင်။

```
[24]: %cd /home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/my-th/evaluation
```

```
/home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/my-th/evaluation
```

```
[25]: !ls * --color=auto
```

```
test.cleaned.1          test.input.txt.1      test.nist-bleu-c.1
test.detokenized.sgm.1  test.multi-bleu.1     test.output.1
test.filtered.ini.1     test.nist-bleu.1      test.output.1.wa

test.analysis.1:
bleu-annotation          n-gram-recall.3
corpus-coverage-by-phrase n-gram-recall.4
corpus-coverage-summary  precision-by-corpus-coverage
```

corpus-unknown	precision-by-input-word
input-annotation	precision-by-ttable-coverage
n-gram-precision.1	segmentation
n-gram-precision.2	segmentation-annotation
n-gram-precision.3	summary
n-gram-precision.4	ttable-coverage-by-phrase
n-gram-recall.1	ttable-coverage-summary
n-gram-recall.2	ttable-unknown

```
test.filtered.1:
info          phrase-table.0-0.1.1.gz
input.4083555 reordering-table.1.wbe-msd-bidirectional-fe.0-0.1
moses.ini
```

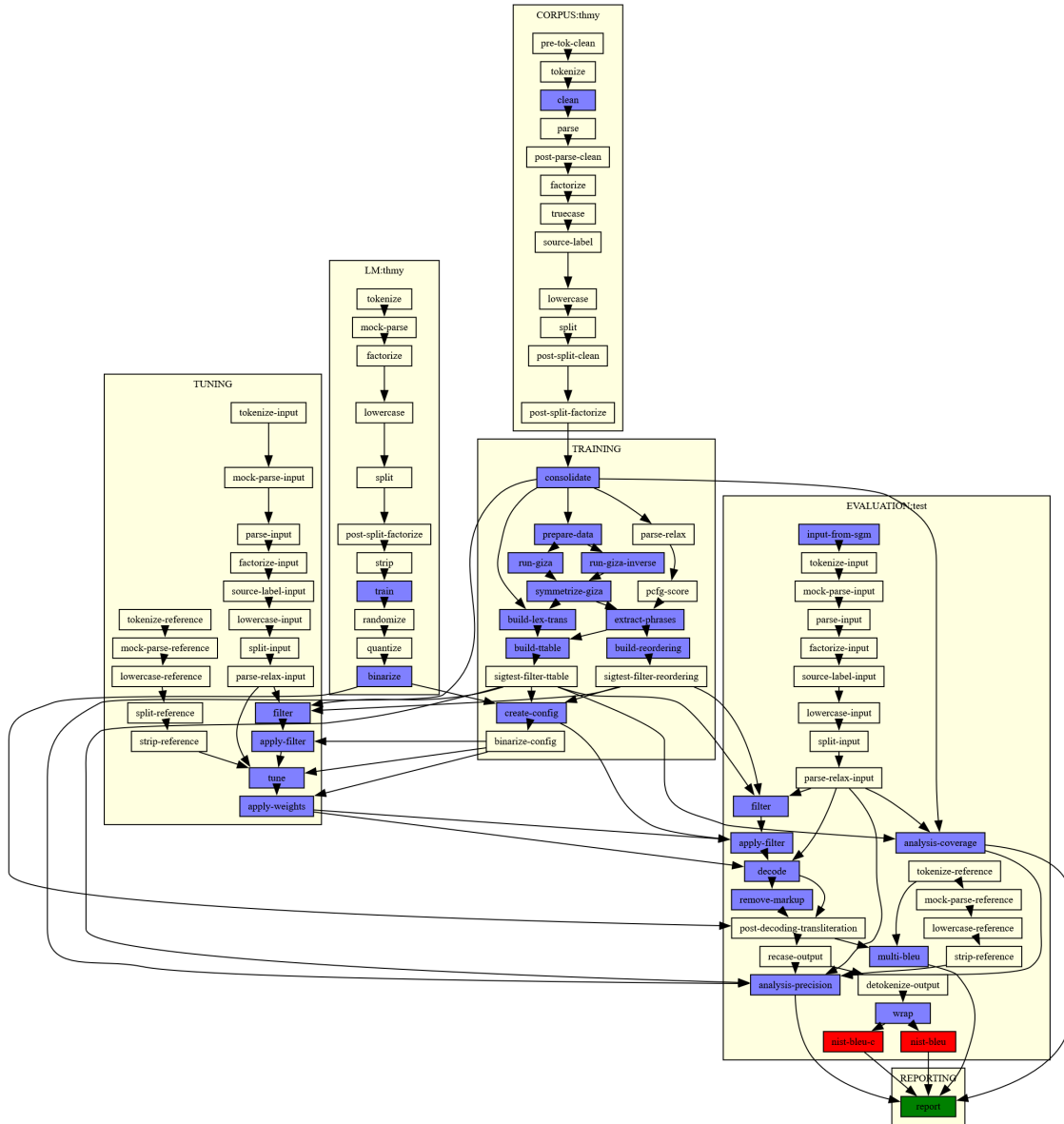
```
[26]: !cat test.multi-bleu.1
```

BLEU = 6.19, 30.7/9.0/3.3/1.6 (BP=1.000, ratio=1.220, hyp_len=858, ref_len=703)

BLEU score က test-run လုပ်ထားတာမို့ တအားနည်းပါတယ်။ SMT pipeline အစအဆုံး ပြီးမပြီး confirm လုပ်ရအောင်။

```
[28]: from IPython.display import Image
Image(filename='/home/ye/exp/SMT-NMT_tutorial/error-examples/graph.1.
↳my-th-2ndtime.png', width=800)
```

```
[28]:
```



အောက်ဆုံးပိုင်းမှာ multi-bleu ပါလာတာကို တွေ့ရပါပြီ။ အဲဒါကြောင့် BLEU score တွက်ပေးထားတာကို မြင်ရတာပါ။ ကောင်းပြီ။ ထိုင်း-မြန်မာ ဘက်ရဲ့ ရလဒ်ကို ဝင်ကြည့်ရအောင်။

```
[29]: %cd /home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/th-my/evaluation
```

```
/home/ye/exp/SMT-NMT_tutorial/pbsmt/baseline/th-my/evaluation
```

```
[30]: !cat ./test.multi-bleu.1
```

```
BLEU = 11.77, 40.4/18.3/8.2/3.8 (BP=0.955, ratio=0.956, hyp_len=1163,
ref_len=1216)
```

ထိုင်း-မြန်မာ ရလဒ်ကတော့ အထက်ပါအတိုင်းပါပဲ။ 11.77 ရရှိပါတယ်။
test-run အစအဆုံး ပြီးပါပြီ။

ခုလောက်ဆိုရင် SMT running pipeline တော့ အကြမ်း နားလည်ပြီလို့ ယူဆပါတယ်။
နောက်ထပ် Jupyter notebook နှစ်ခုမှာတော့ လက်ရှိ ထိုင်းဘက်က Lab မှာ ပြင်ဆင်နေဆဲဖြစ်တဲ့ မြန်မာ-ထိုင်း
medical domain parallel corpus အကုန်ကို သုံးပြီး SMT ထပ် run ပါမယ်။

[]: